

UNIVERSITÉ CLERMONT AUVERGNE

Clermont-Ferrand

École Universitaire de Physique et d'Ingénierie

RAPPORT DE STAGE

1^{ère} année de Master

Spécialité : Mécatronique

Rigon Ajvazi

Détection et estimation 3D d'une balle rapide par caméra événementielle

Période de stage	du 1 ^{er} avril 2026 au 25 juillet 2026
Organisme d'accueil	Institut Pascal, Université Clermont Auvergne / CNRS
Soutien financier	Bourse CIR ITPS, International Research Centre on Innovative Transportation and Production Systems, I-SITE CAP 20-25
Tuteurs de stage	Sébastien Lengagne, Céline Teulière, Antonio Claudio de Sousa dos Santos Filho
Tuteur académique	Benoit Thuilot
Date de soutenance	à compléter

Soutien financier

Remerciements

Je tiens tout d'abord à remercier mon tuteur de stage, Sébastien Lengagne, ainsi que Céline Teulière, pour leur encadrement, leur disponibilité, leur aide et les échanges qui ont orienté mon travail tout au long du stage. Leur accompagnement m'a permis de mieux comprendre les enjeux de la perception robotique rapide et de situer mon travail dans un contexte de recherche appliquée.

Je remercie également mon tuteur académique, Benoit Thuilot, pour le suivi du stage et pour les conseils apportés dans la structuration du travail. Je remercie plus largement les membres de l'Institut Pascal et les personnes avec lesquelles j'ai pu échanger au cours du stage, pour leur accueil.

Enfin, je remercie l'Université Clermont Auvergne et l'Institut Pascal de m'avoir permis de réaliser ce stage dans un cadre associant robotique, vision événementielle et apprentissage par renforcement.

Ce travail a bénéficié du soutien du Centre International de Recherche *Innovative Transportation and Production Systems* (CIR ITPS) de l'I-SITE CAP 20-25, par l'attribution d'une bourse CIR ITPS.

Résumé du sujet de stage

Ce rapport présente le travail réalisé pendant un stage effectué du 1^{er} avril 2026 au 25 juillet 2026 à l'Institut Pascal, unité mixte de recherche de l'Université Clermont Auvergne et du CNRS. Le sujet porte sur la détection et l'estimation 3D d'une balle rapide à partir d'une caméra événementielle DVXplorer, dans la perspective d'une tâche robotique d'interception.

L'objectif du stage est de construire une chaîne de perception capable de transformer le flux d'événements produit par la caméra en une position de balle exploitable par un robot. Cette information doit être suffisamment rapide et cohérente pour préparer une interception, tout en tenant compte des contraintes de calibration, de bruit et de mouvement rapide. Le rapport présente le contexte de l'organisme d'accueil, l'environnement de travail, les choix algorithmiques, les étapes de calibration, les résultats obtenus et les perspectives d'intégration avec la partie robotique.

Mots clés : caméra événementielle, robotique, estimation 3D, détection de balle, ROS 2, Isaac Lab, apprentissage par renforcement.

Table des matières

Remerciements	i
Résumé du sujet de stage	ii
1 Introduction	1
2 Présentation de l'organisme d'accueil	2
2.1 Université Clermont Auvergne	2
2.2 Institut Pascal	3
2.3 Historique et implantation	3
2.4 Activités et axes de recherche	3
2.5 Domaines d'application et partenariats	4
2.6 Centre International de Recherche CIR ITPS	4
2.7 Organisation interne	5
3 Architecture générale du projet	7
3.1 Poste occupé et étude confiée	7
3.2 Environnement technique	7
3.3 Chaîne de traitement	8
3.4 Calibration intrinsèque et repère caméra	9
4 Perception événementielle de la balle	12
4.1 Méthode par circle fitting	12
4.2 Méthode par trace événementielle	22
4.3 Simulation et validation	31
5 Intégration robotique et contrôle	35
5.1 Calibration extrinsèque par mire événementielle	35

5.2	Contrôle robotique : Isaac Lab et PPO	44
	Bibliographie et webographie	46

Chapitre 1

Introduction

Le sujet du stage s'appuie sur une situation simple à décrire : une personne lance une balle vers un robot manipulateur, et le robot doit l'attraper avec un filet à papillon fixé à son effecteur. Pour réussir cette interception, le robot doit connaître très vite la position de la balle, puis utiliser cette information pour placer le filet au bon endroit au bon moment.

Le stage a été réalisé à l'Institut Pascal, au sein de l'Université Clermont Auvergne, du 1^{er} avril 2026 au 25 juillet 2026. Il concerne principalement la partie perception de cette chaîne. La balle est observée avec une caméra événementielle DVXplorer, c'est-à-dire un capteur qui ne fournit pas des images complètes à fréquence fixe, mais des événements datés lorsqu'un changement local de luminosité est détecté. Ce fonctionnement est adapté à une balle rapide, car il donne une information temporelle fine et limite la quantité de données à traiter.

Le problème central est donc de transformer un flux d'événements en une position 3D de balle fiable, avec une latence assez faible pour rester utile à la commande du robot. Une erreur de profondeur ou un retard de quelques millisecondes peut déplacer fortement le point d'interception. La perception doit donc être rapide, robuste au bruit, cohérente dans le temps et compatible avec la calibration entre la caméra et le robot.

Les objectifs généraux du projet sont les suivants :

- obtenir une position 3D exploitable de la balle à partir d'une caméra événementielle ;
- conserver une latence faible pour rendre l'information utilisable pendant le mouvement de la balle ;
- assurer la cohérence entre la perception, la calibration caméra-robot et la commande d'interception ;
- identifier les limites de la chaîne avant son intégration complète sur le robot réel.

Ce rapport commence par présenter l'organisme d'accueil, l'Université Clermont Auvergne et l'Institut Pascal, puis l'architecture générale du projet. La suite est organisée autour de deux grands blocs techniques : la perception événementielle de la balle, puis l'intégration robotique avec la calibration et le contrôle. Le rapport se termine par un bilan technique et personnel du stage.

Chapitre 2

Présentation de l'organisme d'accueil

2.1 Université Clermont Auvergne

L'Université Clermont Auvergne (UCA) est un établissement public expérimental d'enseignement supérieur et de recherche. La nouvelle UCA, sous ce statut, est née le 1^{er} janvier 2021 dans le prolongement de la structuration du site clermontois et de la démarche I-SITE CAP 20-25. Elle regroupe des activités de formation, de recherche et d'innovation sur plusieurs sites à Clermont-Ferrand et dans la région Auvergne.

L'UCA indique rassembler environ 36 000 étudiants, 350 formations et 47 structures de recherche. Elle s'organise autour de six instituts qui regroupent les composantes de formation et de recherche par grands domaines. Cette organisation permet de faire le lien entre enseignement supérieur, laboratoires, plateformes expérimentales et partenaires socio-économiques.

TABLE 2.1 – Carte d'identité synthétique de l'Université Clermont Auvergne et de l'organisme d'accueil.

Élément	Information
Nom de l'établissement	Université Clermont Auvergne (UCA)
Statut	Établissement public expérimental à caractère scientifique, culturel et professionnel
Adresse du siège	49 boulevard François-Mitterrand, CS 60032, 63001 Clermont-Ferrand Cedex 1
Téléphone	04 73 17 79 79
SIRET du siège	130 028 061 00013
Organisme d'accueil du stage	Institut Pascal, UMR 6602 UCA/CNRS
Adresse de l'Institut Pascal	Campus universitaire des Cézeaux, 4 avenue Blaise Pascal, TSA/CS 60026, 63178 Aubière Cedex
Effectif de l'Institut Pascal	Environ 400 personnes
Activité principale	Formation, recherche et innovation, en particulier en sciences de l'ingénierie pour l'Institut Pascal
Capital social	Non applicable pour un établissement public

2.2 Institut Pascal

L'Institut Pascal est l'unité de recherche dans laquelle le stage s'est déroulé. Il s'agit d'une unité mixte de recherche et de formation interdisciplinaire, UMR 6602, placée sous la tutelle de l'Université Clermont Auvergne et du CNRS, avec le CHU de Clermont-Ferrand comme tutelle secondaire. L'institut est également membre de Clermont Auvergne INP. Il couvre les sciences de l'ingénierie du site clermontois : génie des procédés, mécanique, robotique, physique des sciences de l'information et ingénierie de la santé.

L'Institut Pascal rassemble environ 400 personnes et se positionne sur des recherches allant du développement de connaissances fondamentales jusqu'à leur intégration dans des systèmes. Ses domaines d'application principaux sont la mobilité et les transports, l'usine et l'industrie 4.0, ainsi que l'hôpital du futur. Cette orientation explique la place importante donnée aux systèmes robotiques, à la perception, à la commande et aux plateformes expérimentales.

2.3 Historique et implantation

La structuration de l'Institut Pascal s'est faite progressivement à partir de regroupements de laboratoires du site clermontois. L'objectif était de rassembler des compétences complémentaires dans les sciences de l'ingénierie et des systèmes, afin de favoriser des projets interdisciplinaires. Cette logique se retrouve aujourd'hui dans l'organisation de l'institut, où les axes scientifiques couvrent des domaines différents mais reliés par des problématiques communes : modéliser, percevoir, commander, fabriquer et valider expérimentalement.

Le stage a eu lieu sur le campus universitaire des Cézeaux, à Aubière. Ce campus regroupe plusieurs composantes scientifiques et techniques de l'UCA, ainsi que des laboratoires et plateformes utiles aux travaux en robotique. L'implantation sur un même site facilite les échanges entre étudiants, enseignants-chercheurs, doctorants, ingénieurs et personnels techniques. La figure 2.1 situe schématiquement l'Institut Pascal dans cet environnement.

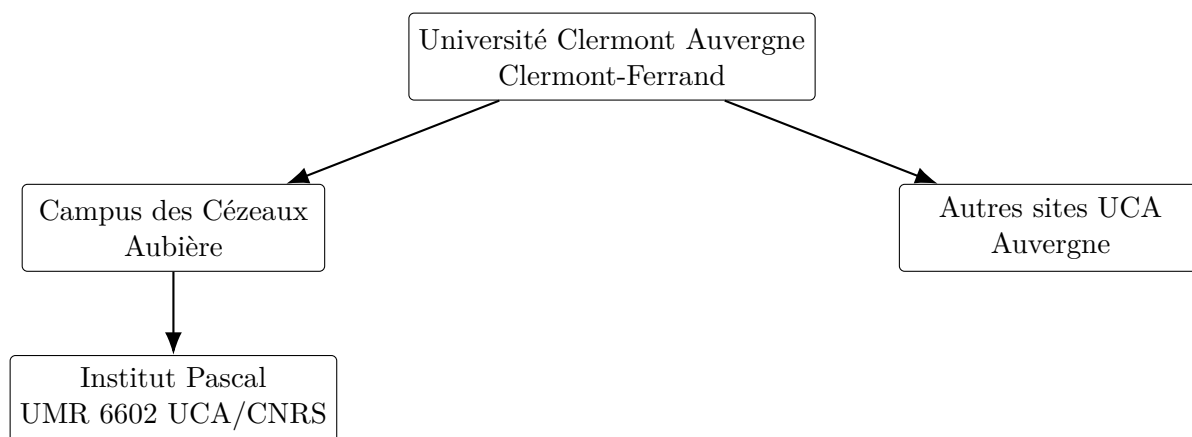


FIGURE 2.1 – Implantation schématique de l'organisme d'accueil dans l'environnement UCA.

2.4 Activités et axes de recherche

Les activités de l'Institut Pascal concernent principalement la recherche académique, les projets collaboratifs, les contrats avec des partenaires industriels, la participation à des réseaux

scientifiques et l'appui à la formation par la recherche.

Ces activités s'appuient sur cinq axes de recherche :

- Génie des Procédés, Énergétique et Biosystèmes (GePEB) ;
- Image, Systèmes de Perception, Robotique (ISPR) ;
- Mécanique, Génie Mécanique, Génie Civil, Génie Industriel (M3G) ;
- Photonique, Ondes, Nanomatériaux (PHOTON) ;
- Thérapies Guidées par l'Image (TGI).

2.5 Domaines d'application et partenariats

Les activités de l'Institut Pascal peuvent être regroupées autour de trois grands domaines d'application. Le premier concerne la mobilité et les transports des biens et des personnes. Il s'agit par exemple de concevoir des systèmes plus sûrs, plus autonomes ou plus efficaces, en s'appuyant sur la mécanique, la robotique, la perception et la modélisation. Le deuxième domaine concerne l'usine et l'industrie 4.0, avec des problématiques de production, d'automatisation, de robotique collaborative, de contrôle et d'interaction humain-machine. Le troisième domaine concerne l'hôpital du futur, dans lequel l'image, les systèmes guidés et les dispositifs d'assistance jouent un rôle important.

TABLE 2.2 – Lecture synthétique des domaines d'application de l'Institut Pascal.

Domaine	Exemples d'enjeux scientifiques et techniques
Mobilité et transports	Perception de l'environnement, robotique mobile, modélisation de systèmes dynamiques, sécurité et autonomie.
Usine et industrie 4.0	Robotique collaborative, automatisation, contrôle, instrumentation, procédés et optimisation industrielle.
Hôpital du futur	Imagerie, dispositifs guidés, assistance au geste, reconstruction 3D et systèmes d'aide à la décision.

Les bénéficiaires et partenaires de ces activités sont des équipes académiques, des établissements publics, des industriels, des collectivités, des doctorants et des étudiants. Les résultats produits prennent la forme de publications scientifiques, de prototypes, de logiciels, de bancs expérimentaux, de méthodes de modélisation, de jeux de données et de transferts vers des partenaires.

L'Institut Pascal participe également à des dispositifs de partenariat structurants. Il est notamment associé à des projets soutenus par des opérateurs de recherche comme l'ANR, l'Union Européenne, la Région Auvergne-Rhône-Alpes ou le programme CAP 20-25. Il est aussi impliqué dans des collaborations avec le monde industriel, par exemple autour de FactoLab, laboratoire commun lié à Michelin et orienté vers l'industrie du futur. Cette proximité avec des problématiques réelles explique que les travaux de recherche ne se limitent pas à des modèles théoriques : ils cherchent souvent à produire des solutions testables sur des plateformes expérimentales.

2.6 Centre International de Recherche CIR ITPS

Le Centre International de Recherche *Innovative Transportation and Production Systems* (CIR ITPS) est l'un des dispositifs de l'I-SITE CAP 20-25. Il a pour rôle de fédérer les activités de

recherche du site clermontois autour des systèmes innovants pour les transports et la production, de renforcer la visibilité internationale du site et de contribuer au déploiement territorial des activités scientifiques et de valorisation [2].

Ses thématiques couvrent plusieurs enjeux liés à la mobilité, à la production et aux systèmes autonomes : autonomie des robots mobiles et des véhicules, développement de cobots mobiles, planification des systèmes de production ou de transport, nouveaux vecteurs énergétiques et acceptabilité des nouvelles technologies [2]. Le CIR ITPS rassemble ainsi des compétences qui vont de la mécanique et de la robotique jusqu'à l'aide à la décision, aux matériaux durables et aux sciences humaines.

Le sujet de ce stage s'inscrit principalement dans la partie robotique et perception de ce cadre. L'objectif final, qui consiste à permettre à un robot d'intercepter une balle rapide, correspond à une problématique de système dynamique où la perception, la commande et l'expérimentation doivent être étudiées ensemble. La caméra événementielle répond au besoin d'une perception rapide, tandis que la partie robotique prépare l'utilisation de cette information pour une action physique.

Le stage présenté dans ce rapport a bénéficié d'une bourse CIR ITPS. Cette aide rattache le travail à un cadre de recherche appliquée où les méthodes développées doivent pouvoir être testées sur des systèmes réels ou proches du réel. La mention de ce soutien et l'utilisation des logos CIR ITPS et I-SITE CAP 20-25 répondent aux consignes officielles applicables aux rapports de stage soutenus par ce centre [3].

2.7 Organisation interne

La structure interne de l'Institut Pascal repose sur des axes scientifiques qui correspondent à de grands ensembles de compétences. Cette organisation permet de regrouper les chercheurs et personnels techniques par disciplines, tout en favorisant des projets transversaux. Dans le cas de mon stage, la perception événementielle mobilise plusieurs notions complémentaires : géométrie 3D, traitement du signal, programmation temps réel, robotique et apprentissage. Le sujet se rattache principalement à l'axe ISPR, avec un lien vers l'équipe MACCS pour la robotique et la commande, et vers l'équipe ComSee pour la vision artificielle. La figure 2.2 donne une lecture simplifiée de cette organisation et de la place du stage.

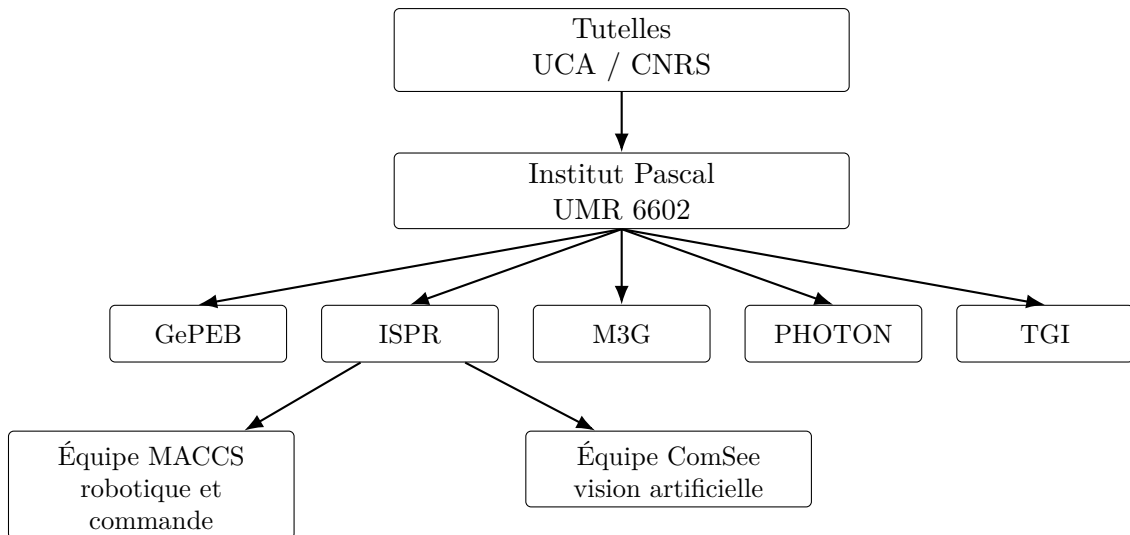


FIGURE 2.2 – Organigramme simplifié situant le stage dans l’organisation scientifique de l’Institut Pascal.

Positionnement du sujet dans l’organisme

Le sujet de stage se situe au croisement de plusieurs objectifs de l’Institut Pascal : développer des méthodes de perception, les tester sur des systèmes rapides, et préparer leur utilisation dans un contexte robotique. Il ne s’agit pas uniquement de détecter une balle dans une image. Il faut produire une estimation 3D robuste, temporellement cohérente, suffisamment rapide et compatible avec une commande robotique.

Ce positionnement justifie l’utilisation d’une caméra événementielle. Pour une tâche d’interception, une caméra classique peut fournir trop peu d’informations temporelles ou introduire une latence trop importante. La caméra événementielle répond à cette difficulté en transmettant uniquement les variations de luminosité, avec une résolution temporelle plus fine. En revanche, elle impose de changer les méthodes de traitement : on ne travaille plus sur une image dense, mais sur un nuage d’événements irrégulier, bruité et dépendant du mouvement.

Le stage contribue donc à une étape intermédiaire essentielle entre la perception brute et la robotique. Les méthodes développées doivent transformer un flux d’événements en une trajectoire exploitable. Cette trajectoire pourra ensuite être calibrée dans le repère du robot, filtrée, prédite et utilisée par une politique d’interception ou un contrôleur. Le sujet est ainsi représentatif d’une problématique de mécatronique : le résultat dépend à la fois du capteur, de l’algorithme, du modèle géométrique, du logiciel temps réel et du système robotique.

Chapitre 3

Architecture générale du projet

3.1 Poste occupé et étude confiée

Pendant le stage, j'ai occupé une fonction de développement et d'expérimentation autour d'une chaîne de perception robotique. Le travail confié consistait à étudier comment détecter une balle rapide dans un flux événementiel, estimer sa position 3D, analyser les limites des méthodes mises en place et préparer l'utilisation de cette information par une partie robotique.

La tâche finale à réaliser est de permettre à un robot à 6 degrés de liberté, ici un UR3e, d'attraper une balle lancée vers lui. Pour détecter la balle avec une latence suffisamment faible, le système s'appuie sur une caméra événementielle, dont les données doivent être transformées en une information de position exploitable par le robot.

Les tâches principales ont été les suivantes :

- comprendre le fonctionnement de la caméra événementielle et le format des données produites ;
- développer une chaîne C++/ROS 2 [6] de filtrage, regroupement et estimation de la balle ;
- comparer une méthode par ajustement de cercle avec une méthode par analyse de trace ;
- utiliser des séquences simulées pour disposer d'une vérité terrain et analyser les erreurs ;
- identifier les besoins de calibration entre la caméra et le robot ;
- préparer le lien avec une politique d'interception apprise en simulation.

3.2 Environnement technique

L'environnement technique du stage combine plusieurs outils matériels et logiciels. La caméra DVXplorer [4] fournit les événements utilisés par les méthodes de détection. Les données peuvent provenir de la caméra réelle ou de séquences enregistrées, ce qui permet de répéter les essais et de comparer plusieurs versions de l'algorithme dans les mêmes conditions. Le traitement est réalisé principalement en C++, car la tâche impose une vitesse de calcul élevée et une latence faible : l'information produite doit rester utilisable pour une interception rapide. ROS 2 [6] est utilisé pour publier les résultats et préparer l'intégration avec la chaîne robotique. Une interface adaptée au projet est également développée avec raylib [8], une bibliothèque C/C++ légère destinée aux affichages graphiques interactifs, afin de visualiser les événements, les détections et les estimations 3D pendant les essais.

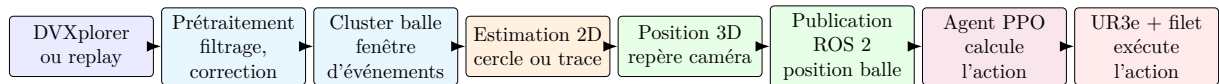
TABLE 3.1 – Principaux moyens techniques mobilisés pendant le stage.

Élément	Utilisation dans le stage
Caméra événementielle DVXplorer [4]	Acquisition d'événements à faible latence pour suivre une balle rapide.
Séquences enregistrées	Relecture des essais pour tester les algorithmes de manière reproductible.
C++ et ROS 2 [6]	Traitement rapide à faible latence, publication des résultats et préparation de l'intégration robotique.
OpenCV [7]	Correction géométrique, calibration intrinsèque et calculs associés à la projection caméra.
raylib [8]	Bibliothèque graphique légère utilisée pour créer une interface adaptée au projet et visualiser les résultats.
Isaac Lab [10] et PPO [13]	Étude de la partie robotique et de l'apprentissage d'une politique d'interception.
Visualisations 2D/3D	Diagnostic des clusters, des trajectoires et des erreurs de profondeur.

Cette combinaison d'outils impose de garder une cohérence entre plusieurs repères et plusieurs échelles de temps. Les événements ont leurs propres timestamps, les estimations 3D sont exprimées dans le repère caméra, le robot travaille dans son repère de base, et la simulation utilise encore ses propres conventions. Une partie importante du travail consiste donc à vérifier que les informations produites par la perception pourront être utilisées sans ambiguïté dans la suite de la chaîne.

3.3 Chaîne de traitement

La chaîne de traitement transforme le flux brut d'événements en une position 3D exploitable par la partie robotique. Les événements proviennent soit de la caméra DVXplorer [4], soit de séquences enregistrées pour répéter les essais. Ils sont d'abord filtrés, corrigés avec la calibration de la caméra, conservés dans une fenêtre temporelle courte puis échantillonnés afin de limiter la charge de calcul. Les points restants sont ensuite regroupés spatialement pour isoler la balle, puis analysés par une méthode d'estimation 2D, par cercle ou par trace. Cette estimation est convertie en position 3D, visualisée dans l'interface et publiée dans ROS 2 [6]. La position est ensuite envoyée à l'agent PPO, qui calcule les actions à appliquer au robot pour placer le filet d'interception.



Principe : ne pas reconstruire une image complète, mais garder les événements utiles pour produire rapidement une position de balle exploitable par la commande robotique.

FIGURE 3.1 – Vue simplifiée de la chaîne de traitement C++/ROS 2 [6], depuis l'estimation 3D de la balle jusqu'à l'action calculée par l'agent PPO.

3.4 Calibration intrinsèque et repère caméra

Cette section présente la calibration intrinsèque utilisée par la chaîne de perception. Elle décrit le modèle optique de la caméra et les paramètres nécessaires pour passer d'une position image à une estimation 3D dans le repère caméra. La calibration entre la caméra et le robot est traitée plus loin, dans la partie consacrée à l'intégration robotique.

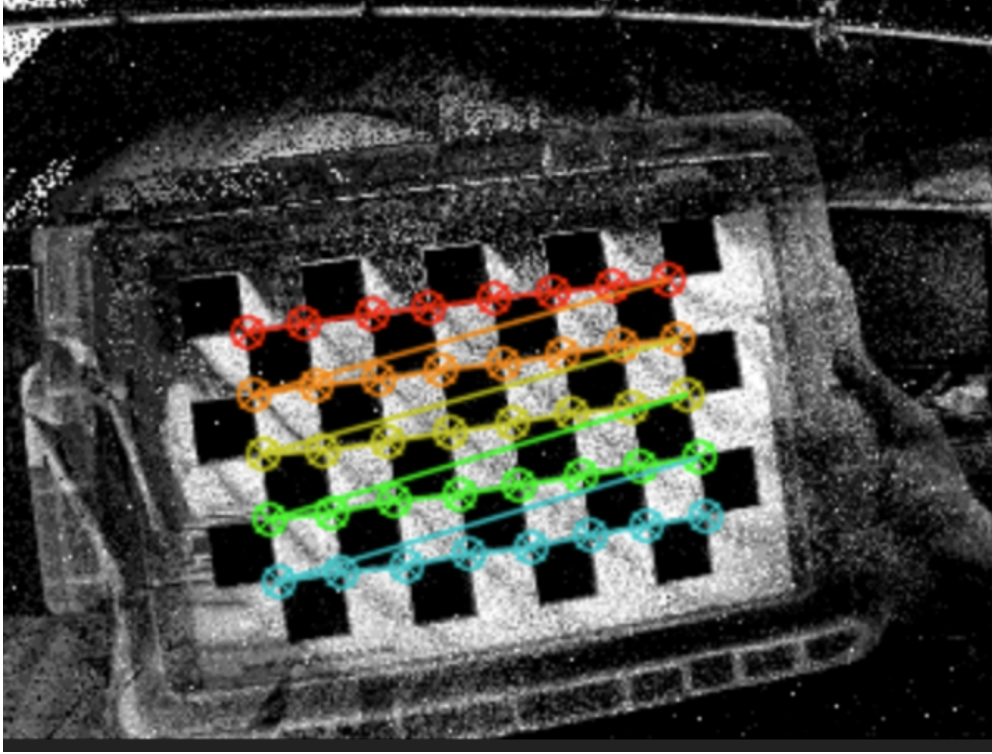


FIGURE 3.2 – Calibration intrinsèque de la caméra événementielle.

La première calibration intrinsèque de la caméra réelle donne les paramètres suivants :

$$f_x \approx 761 \text{ px}, \quad f_y \approx 754 \text{ px}, \quad c_x \approx 305 \text{ px}, \quad c_y \approx 196 \text{ px}.$$

Cette première calibration a été réalisée avec les outils officiels d'iniVation [5]. Comme la caméra événementielle ne fournit pas directement une image complète, il a fallu bouger constamment la mire d'échiquier devant la caméra afin de générer assez d'événements pour reconstruire une image exploitable. L'erreur de calibration obtenue est d'environ 0.486 px.

Le bruit observé avec cette méthode a ensuite motivé la création d'un nouvel outil Python, avec un principe de calibration pensé directement pour une caméra événementielle. Au lieu de chercher les coins d'un échiquier dans une image reconstruite, la méthode utilise une mire affichée sur un écran, qui produit volontairement des variations de luminosité. Les disques lumineux clignotent sur fond noir, ce qui génère des événements nets aux positions utiles. Cette approche est aussi plus simple à mettre en œuvre et moins contraignante expérimentalement, car elle évite de devoir déplacer continuellement une mire physique pour produire suffisamment d'événements. Elle a également réduit de manière importante le temps nécessaire pour réaliser une calibration exploitable.

Le principe est séparé en deux étapes. Le premier programme Python affiche une mire de 4×5

disques, avec un disque manquant et un disque plus grand servant d'ancre. Cela donne 19 points détectables et limite les ambiguïtés d'orientation. Pendant une courte fenêtre temporelle, les événements positifs et négatifs sont accumulés dans une image d'activité. Les zones actives sont ensuite seuillées, regroupées en composantes connexes, puis chaque centre de disque est estimé par barycentre pondéré de l'activité. Comme la taille physique de l'écran est connue, chaque disque possède aussi une coordonnée 3D connue dans le plan de la mire, exprimée en millimètres. Chaque acquisition fournit alors les correspondances entre les points de la mire $(X, Y, 0)$ et leurs centres image (u, v) .

Le second programme Python regroupe ces observations et lance la calibration OpenCV [7]. Il estime la matrice intrinsèque, les coefficients de distorsion et l'erreur de reprojection. Une sélection robuste peut être activée : plusieurs sous-ensembles d'observations sont testés, les vues cohérentes sont conservées, puis la calibration finale est relancée sur les vues retenues. Cette étape permet d'écarter automatiquement des captures où l'association de points est moins fiable.

Dans l'essai conservé, la calibration robuste utilise 32 observations de mire sur une image 640×480 , avec une erreur RMS d'environ 0.189 px. Elle produit notamment :

$$f_x \approx 526.3 \text{ px}, \quad f_y \approx 525.6 \text{ px}, \quad c_x \approx 330.9 \text{ px}, \quad c_y \approx 242.2 \text{ px}.$$

Ces valeurs ne doivent pas être comparées directement à celles de la calibration par échiquier présentée plus haut. Entre les deux essais, la monture de la caméra a été touchée, ce qui a pu modifier le réglage mécanique et donc les paramètres observés. L'écart entre les deux calibrations ne permet donc pas, à lui seul, de conclure qu'une méthode est meilleure que l'autre ; il montre surtout que les deux acquisitions ne correspondent pas exactement au même état matériel.

Les coefficients de distorsion radiale estimés sont environ $k_1 = -0.403$ et $k_2 = 0.167$, avec une distorsion tangentielle fixée à zéro et k_3 fixé à zéro. Le résultat est ensuite sauvegardé dans un format compatible avec la chaîne de suivi de balle.

Un test complémentaire a été ajouté pour vérifier que cette calibration ne fonctionne pas seulement sur la mire utilisée pour l'estimation. Une cible carrée à quatre points est affichée dans plusieurs positions et tailles, puis les distances reconstruites sont comparées aux dimensions réelles de l'écran. Sur l'exemple retenu, les quatre carrés sont validés, avec une erreur RMS d'environ 0.13 px et une erreur moyenne d'espacement d'environ 0.16 mm. La figure 3.3 montre le montage utilisé et une sortie de validation.

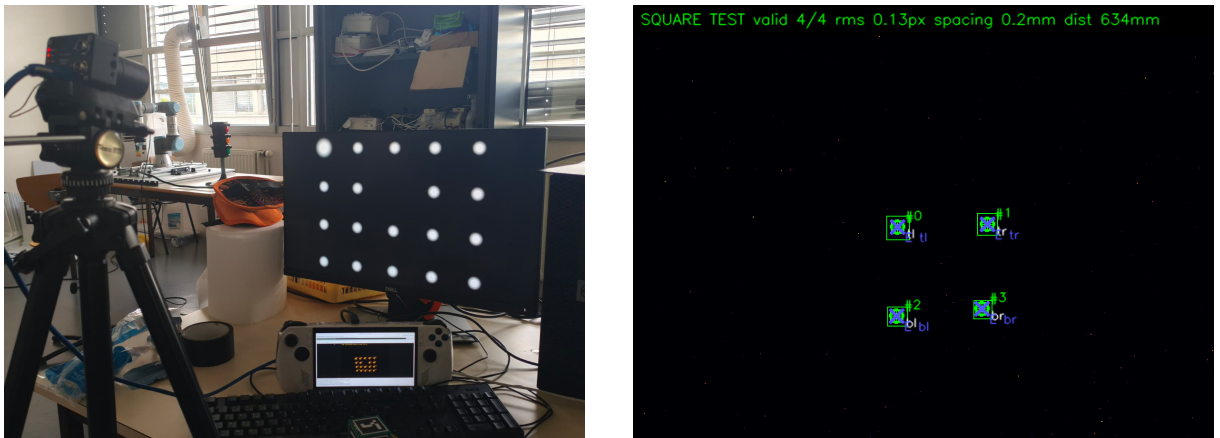


FIGURE 3.3 – Calibration intrinsèque par mire événementielle. À gauche, la caméra observe une mire clignotante affichée sur écran ; à droite, le test carré vérifie la cohérence de la calibration obtenue.

Pour les essais en simulation, la caméra virtuelle est définie avec une calibration synthétique différente :

$$f_x = f_y = 520 \text{ px}, \quad c_x = 320 \text{ px}, \quad c_y = 240 \text{ px}.$$

Chapitre 4

Perception événementielle de la balle

4.1 Méthode par circle fitting

4.1.1 Signal événementiel et polarités

Une caméra événementielle ne transmet pas des images complètes à fréquence fixe. Chaque pixel fonctionne de manière indépendante et déclenche un événement lorsqu’il détecte une variation suffisante de luminosité. Un événement contient donc quatre informations principales :

$$e = (x, y, t, p),$$

où x et y sont les coordonnées du pixel, t est le timestamp, et p est la polarité. La polarité indique le signe de la variation de luminosité : une polarité positive correspond à une augmentation locale de luminosité, tandis qu’une polarité négative correspond à une diminution. Pour une balle en mouvement, ces polarités apparaissent souvent de manière structurée autour du contour de l’objet. Elles peuvent donc aider à vérifier qu’un groupe d’événements ressemble bien au passage d’une balle plutôt qu’à du bruit isolé.

Ce fonctionnement est particulièrement intéressant pour une balle rapide. La caméra ne produit des données que là où quelque chose change, ce qui réduit fortement la quantité d’information à traiter. En revanche, si l’on accumule trop longtemps les événements, la balle ne forme plus un contour circulaire instantané : elle laisse une trace allongée. L’algorithme doit donc conserver assez d’événements pour être robuste, mais pas au point de mélanger plusieurs positions de la balle.

4.1.2 Pipeline de l’algorithme

La méthode par *circle fitting* considère que la balle est une sphère dont la projection dans l’image peut être approchée par un cercle. L’idée est de détecter les événements appartenant probablement à la balle, d’ajuster un cercle sur ces points, puis d’utiliser le rayon apparent pour calculer la profondeur. La figure 4.1 résume les étapes principales de cette chaîne de traitement.

Algorithme de détection 3D par circle fitting

Traitement d'un flux d'événements pour détecter une balle, estimer sa pose 3D et reconstruire sa trajectoire.

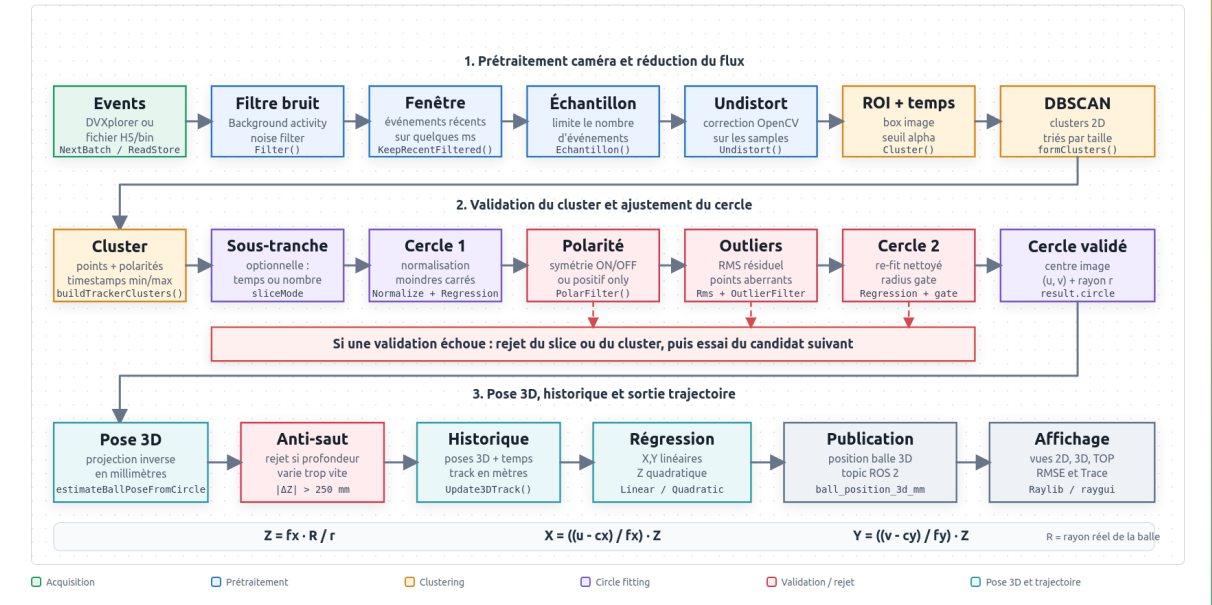


FIGURE 4.1 – Pipeline général de l'algorithme de détection 3D par circle fitting.

Le premier choix important est l'utilisation d'une fenêtre temporelle récente. Cette fenêtre limite les événements analysés aux derniers instants observés par la caméra. Elle réduit le bruit accumulé, évite de mélanger des positions trop anciennes de la balle, et donne une estimation plus réactive. La taille de cette fenêtre est donc un compromis : trop courte, elle ne contient pas assez de points ; trop longue, elle transforme le contour de la balle en traînée.

Après ce fenêtrage, les événements sont échantillonnés. Cette étape ne change pas le modèle géométrique, mais elle est importante pour les performances. DBSCAN doit comparer les points entre eux pour former des groupes denses : si trop d'événements sont conservés, le temps de calcul augmente rapidement. L'échantillonnage garde un sous-ensemble représentatif du flux, ce qui permet de conserver une boucle rapide tout en laissant assez de points pour reconnaître la balle.

L'étape suivante est l'*undistortion*. Les coordonnées image sont corrigées avec la calibration OpenCV [7] afin de compenser la distorsion optique de la caméra. Cette correction est nécessaire avant le clustering final et surtout avant la pose 3D, car le calcul de profondeur suppose que les points suivent le modèle de projection calibré. Dans le pipeline utilisé, l'undistortion est appliquée après l'échantillonnage afin de corriger moins de points et donc de réduire le coût de calcul.

DBSCAN est ensuite utilisé pour regrouper les événements en clusters. Ce choix est adapté au problème car le nombre d'objets visibles n'est pas connu à l'avance et le flux contient du bruit. DBSCAN cherche des zones denses dans l'image, rejette les points isolés, et fournit plusieurs clusters candidats. Chaque cluster peut correspondre à la balle, à une partie de la scène en mouvement, ou à un bruit structuré ; l'étape de circle fitting sert ensuite à sélectionner le candidat le plus cohérent.

4.1.3 Sous-tranches temporelles

Lorsqu'un cluster contient des événements répartis sur une durée trop longue, il peut représenter plusieurs positions successives de la balle. Pour limiter cet effet, l'algorithme peut découper le cluster en sous-tranches. Ces sous-tranches peuvent être définies par fenêtre temporelle ou par nombre d'événements. L'objectif est de retrouver une forme plus proche d'un cercle instantané, au lieu d'ajuster un cercle sur toute la trace. La figure 4.2 illustre une sortie 2D utilisée pour contrôler ce choix.

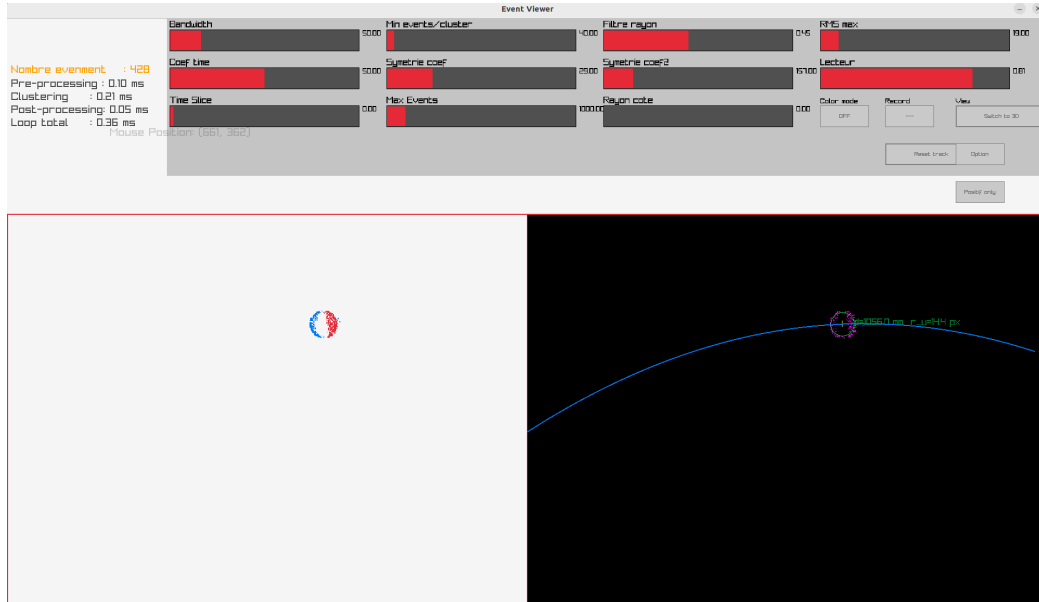


FIGURE 4.2 – Exemple de visualisation 2D utilisée pour contrôler les événements conservés, le cercle détecté et la trajectoire estimée.

Cette étape est particulièrement utile lorsque la balle se déplace vite dans le plan image. Une sous-tranche récente donne une estimation plus proche de la position actuelle, tandis qu'un cluster complet peut être influencé par des événements plus anciens. Elle prépare donc un nuage de points plus stable pour l'ajustement du cercle.

4.1.4 Ajustement du cercle et validation

Le cercle estimé fournit trois informations principales :

$$(u, v, r),$$

où u et v sont les coordonnées du centre dans l'image et r est le rayon apparent en pixels. L'algorithme effectue deux ajustements successifs. Le premier cercle est calculé sur le cluster candidat afin d'obtenir une estimation initiale du centre et du rayon. Cette estimation sert ensuite à analyser les polarités, calculer le résidu RMS et éliminer les points aberrants.

La validation par symétrie est une étape importante de première identification de la balle. Après le premier cercle, le centre estimé sert de référence pour regarder comment les événements positifs et négatifs sont répartis autour du candidat. Pour chaque événement p_i , on calcule le vecteur radial normalisé :

$$d_i = \frac{p_i - c}{\|p_i - c\|},$$

où c est le centre du cercle estimé. Les vecteurs sont ensuite séparés selon leur polarité. L'algorithme accumule les directions des événements positifs et négatifs :

$$D_+ = \sum_{p_i=+} d_i, \quad D_- = \sum_{p_i=-} d_i.$$

Un vrai passage de balle produit généralement deux groupes de polarités opposés : un bord de la balle provoque une augmentation de luminosité et l'autre une diminution. L'algorithme vérifie donc d'abord que les deux polarités sont présentes et que leurs nombres restent relativement équilibrés :

$$\text{sym} = \frac{|N_+ - N_-|}{N_+ + N_-}.$$

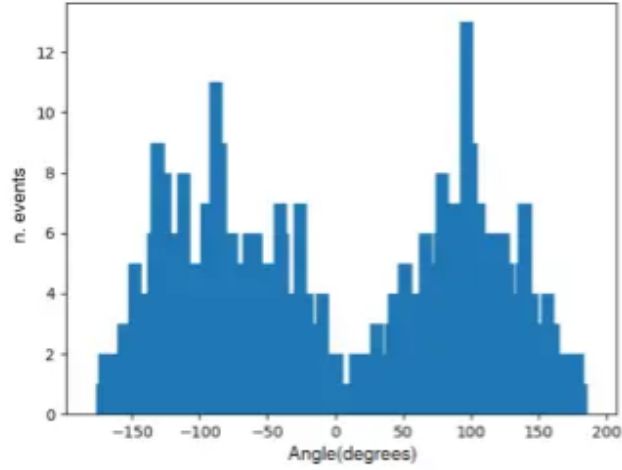
Si cette valeur est trop grande, le cluster est rejeté car il ressemble davantage à du bruit ou à un objet non symétrique qu'à une balle. Ensuite, l'algorithme calcule l'angle moyen de chaque polarité :

$$\theta_+ = \text{atan2}(D_{+,y}, D_{+,x}), \quad \theta_- = \text{atan2}(D_{-,y}, D_{-,x}).$$

La séparation entre ces deux directions doit être proche de π , c'est-à-dire que les deux groupes doivent être approximativement opposés :

$$||\theta_+ - \theta_-| - \pi| < \varepsilon.$$

Cette étape ne donne pas encore la pose 3D finale, mais elle permet d'écarter tôt les clusters qui ne possèdent pas la structure attendue d'une balle en mouvement. Elle rend donc le choix du cluster plus robuste avant le filtrage des points aberrants et le second ajustement de cercle. La figure 4.3, issue du rapport de stage d'Antonio [1], sert ici de schéma explicatif pour visualiser cette séparation des polarités.



(d) Distribution of all events over different angles.

FIGURE 4.3 – Schéma explicatif issu du rapport de stage d’Antonio [1] : les événements positifs et négatifs doivent former deux directions moyennes opposées autour du centre du cercle candidat.

Le second cercle est recalculé après ce nettoyage. Cette deuxième régression est plus fiable car elle s’appuie sur un nuage de points déjà filtré. Le but n’est pas de complexifier inutilement l’algorithme, mais d’éviter qu’un petit nombre de points isolés élargisse artificiellement le rayon. Cette précaution est importante, car la profondeur dépend directement de ce rayon apparent. La figure 4.4 montre la sortie 2D de l’algorithme après clustering, validation et estimation du cercle.



FIGURE 4.4 – Sortie 2D de l’algorithme. La partie gauche montre les événements bruts, tandis que la partie droite montre les clusters, le cercle retenu et la trajectoire image. La flèche verte indique l’estimation de la direction de la balle ; cette direction est stabilisée par la validation de polarité et de symétrie du cluster.

4.1.5 Détermination de la pose 3D

En connaissant le rayon réel R de la balle et la focale f_x , la profondeur est estimée à partir du rayon apparent :

$$Z = \frac{f_x R}{r}.$$

La position 3D complète est ensuite calculée par projection inverse :

$$X = \frac{u - c_x}{f_x} Z, \quad Y = \frac{v - c_y}{f_y} Z.$$

Dans le projet, la balle utilisée dans les séquences simulées possède un rayon de 20 mm. Le même ordre de grandeur est utilisé pour l'estimation réelle. La pose obtenue est exprimée en millimètres dans le repère caméra, puis convertie en mètres pour l'affichage 3D.

La trajectoire représentée sur la figure 4.5 correspond à de bonnes conditions de détection : le cluster de la balle est bien isolé, le cercle ajusté reste cohérent avec la projection de la balle et le rayon apparent n'est pas trop perturbé par une traînée événementielle. Elle illustre donc le comportement attendu de la méthode lorsque le signal respecte bien l'hypothèse circulaire.

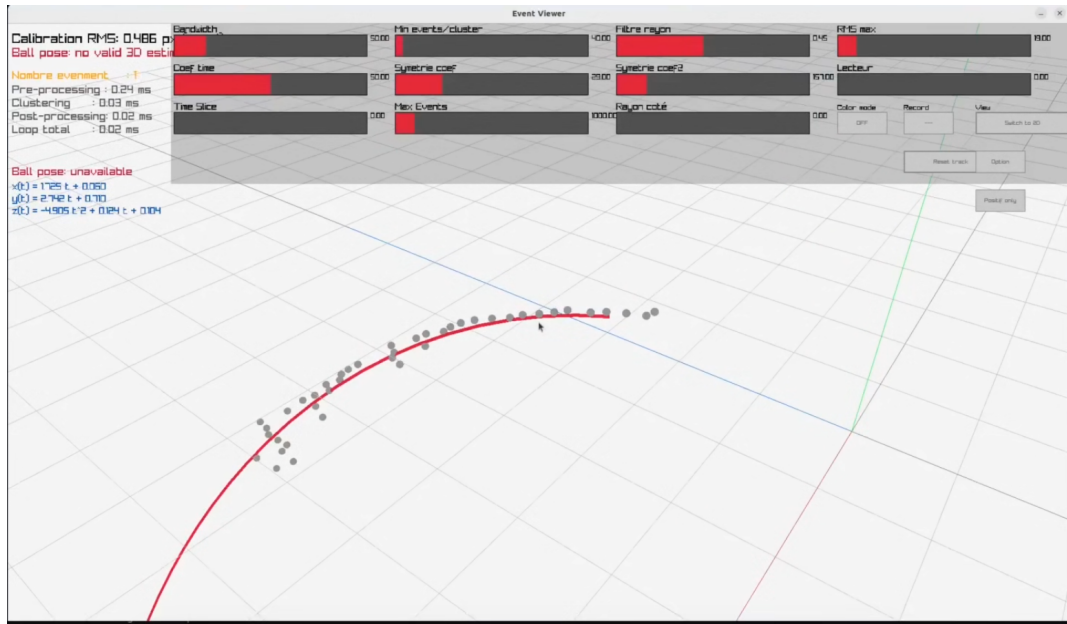


FIGURE 4.5 – Estimation 3D obtenue dans de bonnes conditions de détection, à partir du centre et du rayon du cercle détecté dans l'image.

4.1.6 Régression de trajectoire 3D

Chaque position 3D valide est ajoutée à l'historique. La trajectoire est ensuite modélisée par des régressions simples :

$$X(t) = a_x t + b_x, \quad Y(t) = a_y t + b_y, \quad Z(t) = a_z t^2 + b_z t + c_z.$$

Pour une coordonnée linéaire $q \in \{X, Y\}$, les coefficients sont calculés directement à partir des sommes accumulées :

$$D = n \sum t_i^2 - \left(\sum t_i \right)^2,$$

$$a = \frac{n \sum t_i q_i - \left(\sum t_i \right) \left(\sum q_i \right)}{D}, \quad b = \frac{\sum q_i - a \sum t_i}{n}.$$

Pour la coordonnée quadratique Z , le modèle est :

$$q(t) = at^2 + bt + c.$$

Les grandeurs intermédiaires utilisées sont :

$$\begin{aligned} S_{11} &= \sum t_i^2 - \frac{(\sum t_i)^2}{n}, & S_{12} &= \sum t_i^3 - \frac{(\sum t_i)(\sum t_i^2)}{n}, \\ S_{22} &= \sum t_i^4 - \frac{(\sum t_i^2)^2}{n}, & S_{q1} &= \sum t_i q_i - \frac{(\sum q_i)(\sum t_i)}{n}, \\ S_{q2} &= \sum t_i^2 q_i - \frac{(\sum q_i)(\sum t_i^2)}{n}, & D_q &= S_{22}S_{11} - S_{12}^2. \end{aligned}$$

Les coefficients quadratiques sont alors :

$$\begin{aligned} b &= \frac{S_{q1}S_{22} - S_{q2}S_{12}}{D_q}, & a &= \frac{S_{q2}S_{11} - S_{q1}S_{12}}{D_q}, \\ c &= \bar{q} - b\bar{t} - a\bar{t}^2. \end{aligned}$$

Une version pondérée peut aussi être activée. Elle garde les mêmes modèles, mais remplace les sommes simples par des sommes pondérées. Le poids d'un point combine un terme de récence et un terme robuste basé sur le résidu :

$$w_i = \exp\left(-3 \frac{t_{\max} - t_i}{t_{\max} - t_{\min}}\right) \cdot \frac{1}{1 + \left(\frac{r_i}{s}\right)^2},$$

où r_i est le résidu par rapport à la première régression et s est une échelle robuste calculée à partir de la médiane des résidus. Les points récents et cohérents ont donc plus d'influence que les anciens points ou les mesures aberrantes.

Ce choix correspond au mouvement attendu d'un projectile : deux axes sont modélisés de manière linéaire, tandis que l'axe soumis principalement à la gravité est modélisé par une parabole. L'objectif n'est pas seulement d'afficher les points mesurés ; la trajectoire sert surtout de filtre temporel pour stabiliser la suite de positions 3D et vérifier que les poses successives restent cohérentes.

La méthode n'utilise pas une résolution matricielle générale pour ces régressions à chaque boucle. Elle accumule directement les sommes nécessaires aux modèles linéaire et quadratique. Ce choix est volontaire : la boucle doit rester rapide, et ces modèles simples peuvent être mis à jour avec peu d'opérations. Construire et résoudre une matrice complète à chaque itération serait plus général, mais moins adapté à une application temps réel où la forme du modèle est déjà connue.

La position de la balle est envoyée en boucle sur le topic ROS 2 [6] à partir des poses 3D validées. La trajectoire 3D sert donc principalement à lisser et filtrer la perception, plutôt qu'à remplacer la mesure instantanée par une prédiction longue.

4.1.7 Limite principale : sensibilité du rayon

La faiblesse de cette méthode vient du fait que toute la profondeur dépend du rayon apparent r . Une petite erreur sur r produit une grande erreur sur Z , surtout lorsque la balle est éloignée et que son rayon en pixels est faible.

À partir de :

$$Z = \frac{f_x R}{r},$$

on obtient la sensibilité :

$$\frac{dZ}{dr} = -\frac{f_x R}{r^2} = -\frac{Z}{r}.$$

Cela signifie qu'une erreur de 1 px sur le rayon produit environ Z/r d'erreur sur la profondeur.

TABLE 4.1 – Sensibilité de la profondeur pour la caméra réelle, avec $f_x \approx 761$ px et $R = 20$ mm.

Z	r image	Largeur	Erreur 1 px rayon	Erreur 1 px largeur
1.00 m	15.23 px	30.45 px	6.6 cm	3.3 cm
1.47 m	10.36 px	20.71 px	14.2 cm	7.1 cm
2.00 m	7.61 px	15.23 px	26.3 cm	13.1 cm

À 1.47 m, le rayon apparent vaut seulement environ 10.36 px. Une erreur d'un pixel sur ce rayon représente donc environ 14 cm d'erreur de profondeur. Si l'on raisonne sur une largeur ou un diamètre, l'erreur d'un pixel correspond à environ 7 cm. Cette sensibilité explique pourquoi le circle fitting devient insuffisant pour une balle rapide.

En pratique, la caméra événementielle ne voit pas toujours un contour circulaire net. Lors d'un mouvement rapide, les événements s'étalent dans le temps et forment une traînée. Le cercle ajusté peut alors surestimer le rayon apparent. Comme Z est inversement proportionnel à r , un rayon surestimé donne une balle artificiellement trop proche.

4.1.8 Décalage du plan estimé à haute vitesse

Un effet visible apparaît lorsque l'on lance une balle de ping-pong dans un plan bien défini devant la caméra. Physiquement, la trajectoire devrait appartenir à un plan réaliste, approximativement perpendiculaire au sol. En vue de dessus, ce plan peut être représenté par une droite. Or les estimations 3D issues du circle fitting ne restent pas toujours sur cette droite : elles peuvent être décalées vers une profondeur plus faible. La figure 4.6 donne deux exemples de ce décalage.

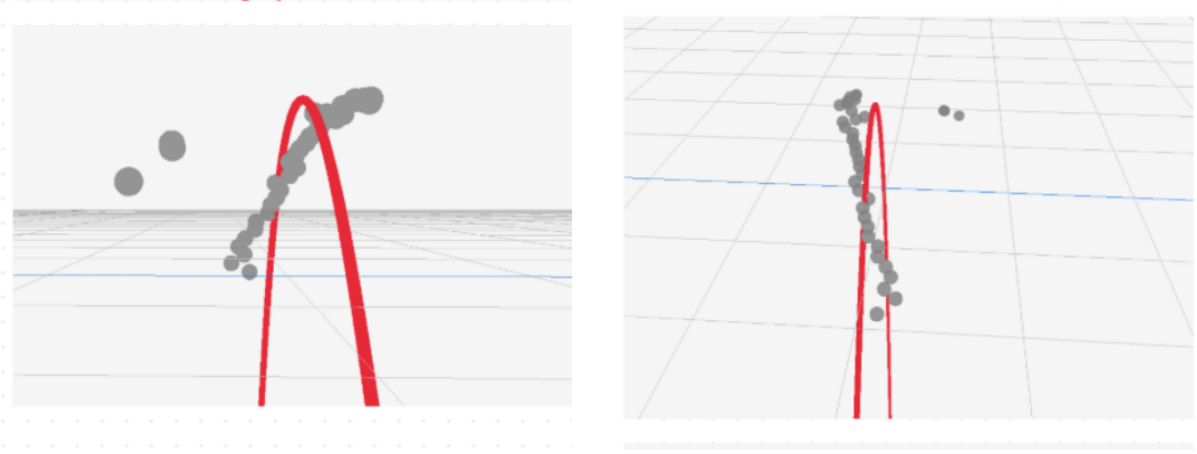


FIGURE 4.6 – Exemples de décalage de la trajectoire 3D estimée lorsque la balle est rapide. Les points estimés ne suivent pas toujours le plan vertical attendu pour le lancer.

La cause principale est la même que précédemment : à haute vitesse, la traînée événementielle devient plus grande. Le cercle ajusté peut alors englober une zone trop large, ce qui augmente le rayon apparent r . Comme la profondeur est calculée avec $Z = f_x R/r$, une surestimation de r produit directement une profondeur trop faible. Les points 3D sont alors ramenés artificiellement vers la caméra, ce qui décale le plan estimé par rapport au plan réel du mouvement.

Pour vérifier si cette erreur peut être corrigée, on peut chercher une relation entre la vitesse de la balle et l'erreur de profondeur. Dans l'outil de visualisation, la vue de dessus ajuste d'abord une droite aux estimations 3D projetées dans le plan X/Y :

$$Y = aX + b.$$

Cette droite représente le plan vertical attendu vu du dessus. Pour chaque estimation $P_i = (X_i, Y_i, Z_i)$, l'outil calcule ensuite la distance orthogonale à cette droite :

$$d_i = \frac{|aX_i - Y_i + b|}{\sqrt{a^2 + 1}}.$$

La vitesse utilisée pour comparer cette erreur est la vitesse dans le plan caméra X/Z , calculée à partir de la régression 3D :

$$v_{XZ}(t) = \sqrt{a_X^2 + (2a_Z t + b_Z)^2}.$$

La distance d_i est alors tracée en fonction de $v_{XZ}(t_i)$, puis une régression linéaire est ajustée :

$$d \approx \alpha v_{XZ} + \beta.$$

Si une proportionnalité apparaît, cela indique que le biais de profondeur dépend de la vitesse. Cette relation pourrait ensuite servir à corriger soit le rayon estimé, soit directement la profondeur, par exemple en ajoutant une correction dépendante de la vitesse. La figure 4.7 présente cette analyse dans l'outil de visualisation.

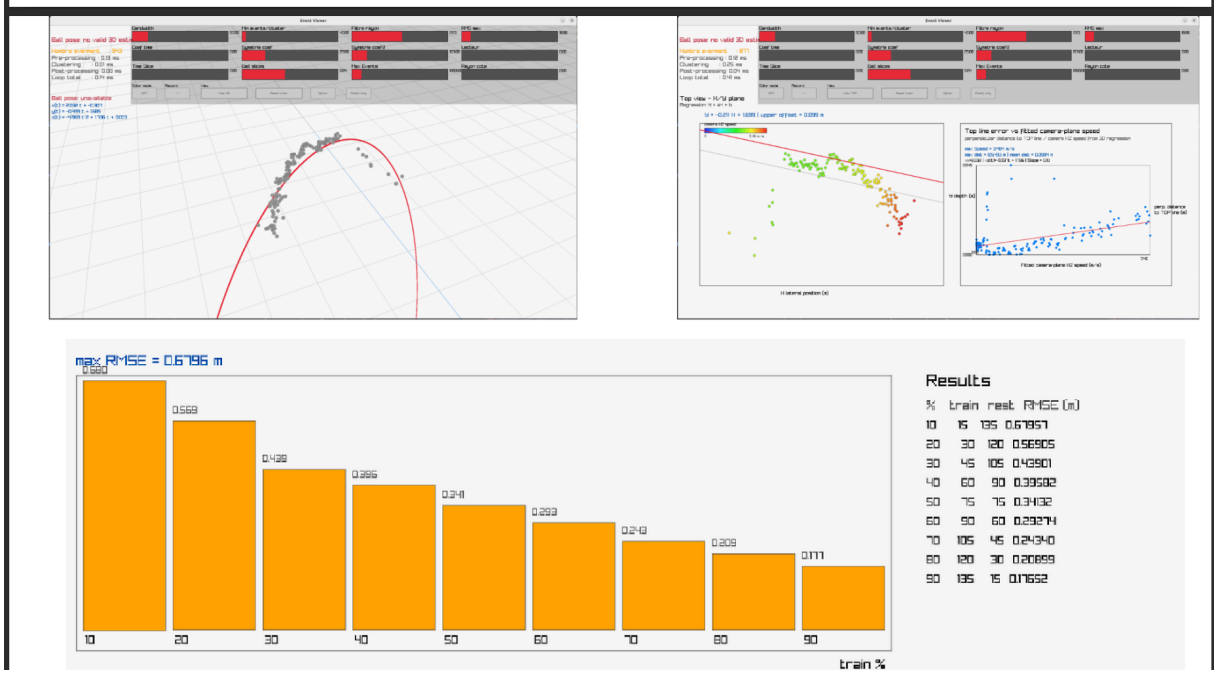


FIGURE 4.7 – Analyse du biais de profondeur. En haut à droite, la vue de dessus montre le décalage des estimations par rapport au plan attendu ; le graphique adjacent trace la distance orthogonale à ce plan en fonction de la vitesse estimée dans le plan caméra. La tendance linéaire suggère une proportionnalité entre vitesse et erreur de profondeur.

La partie basse de cette figure correspond à la vue RMSE de l'outil de visualisation. Pour chaque pourcentage p entre 10% et 90%, l'algorithme utilise les $p\%$ premiers points 3D pour ajuster une trajectoire :

$$X(t) = a_X t + b_X, \quad Y(t) = a_Y t + b_Y, \quad Z(t) = a_Z t^2 + b_Z t + c_Z.$$

Il teste ensuite cette trajectoire sur les points restants, c'est-à-dire les $(100-p)\%$ derniers points. Si N_p est le nombre de points utilisés pour l'apprentissage, la mesure affichée est :

$$\text{RMSE}(p) = \sqrt{\frac{1}{N - N_p} \sum_{i=N_p}^{N-1} \|\hat{P}_i^{(p)} - P_i\|^2}.$$

Cette RMSE ne compare pas directement l'estimation à une vérité terrain externe. Elle mesure plutôt la cohérence interne de la trajectoire estimée : on regarde si les premiers points permettent de prédire correctement la suite des estimations. Lorsque le biais de profondeur varie avec la vitesse, cette erreur augmente car la trajectoire estimée devient moins compatible avec un modèle balistique simple.

4.1.9 Conclusion sur la méthode par circle fitting

La méthode par circle fitting donne une base simple et efficace lorsque la balle produit un contour événementiel suffisamment circulaire. Elle permet d'obtenir rapidement une pose 3D à partir de peu d'informations : un centre image, un rayon apparent et la calibration caméra. Cependant, sa limite principale est structurelle : toute la profondeur dépend directement du rayon estimé.

Or ce rayon est lui-même très sensible au bruit, à la densité d'événements, au choix de la fenêtre temporelle et surtout à la vitesse de la balle.

Lorsque la balle se déplace vite, la traînée événementielle augmente le rayon apparent. La profondeur calculée devient alors trop faible, ce qui décale la trajectoire 3D. On peut essayer de corriger ce biais, par exemple en cherchant une relation entre vitesse et erreur de profondeur, puis en appliquant une compensation. Cette piste est possible, mais elle devient rapidement laborieuse : il faut estimer un coefficient fiable, vérifier qu'il reste valable sur plusieurs lancers, et tenir compte de facteurs difficiles à contrôler.

Parmi ces facteurs, on peut citer la luminosité de la scène, le contraste de la balle, l'angle de tir, la distance à la caméra, la rotation éventuelle de la balle, la densité d'événements produite par la caméra et les erreurs de calibration. Certains de ces effets sont difficiles à mesurer directement, et d'autres peuvent varier d'une séquence à l'autre. Corriger uniquement le rayon risque donc de produire une solution fragile, très dépendante des conditions expérimentales.

Il est donc plus intéressant de chercher une méthode moins sensible à une erreur locale sur le rayon. C'est la motivation de la méthode Trace. Au lieu de forcer un cercle sur une forme parfois allongée, elle exploite la forme réellement observée dans le flux événementiel. La largeur de la trace est estimée à partir de plusieurs coupes et de nombreux événements, ce qui réduit l'influence d'un point isolé ou d'un cercle mal ajusté. Cette approche est plus prometteuse pour les balles rapides, car elle utilise directement l'information produite par la caméra événementielle au lieu de la ramener à tout prix à un contour circulaire instantané.

4.2 Méthode par trace événementielle

4.2.1 Motivation

La méthode par trace part d'une observation simple : pour une balle rapide, l'information utile n'est pas seulement contenue dans un cercle instantané. Elle est répartie dans toute la trace événementielle produite pendant le déplacement de la balle. Cette trace contient beaucoup plus de points qu'un seul contour local, et elle correspond mieux à ce que la caméra événementielle observe réellement lorsque la balle traverse vite le champ de vision.

L'idée est donc de mesurer la largeur réelle de la trace, puis d'utiliser cette largeur comme diamètre apparent de la balle. La profondeur n'est plus déduite d'un rayon de cercle ajusté sur quelques événements, mais d'un ruban estimé à partir de nombreuses coupes locales. Cette approche est plus robuste face aux points isolés, aux contours incomplets et aux déformations créées par la vitesse. La figure 4.8 montre la trace test utilisée pour valider ce principe.

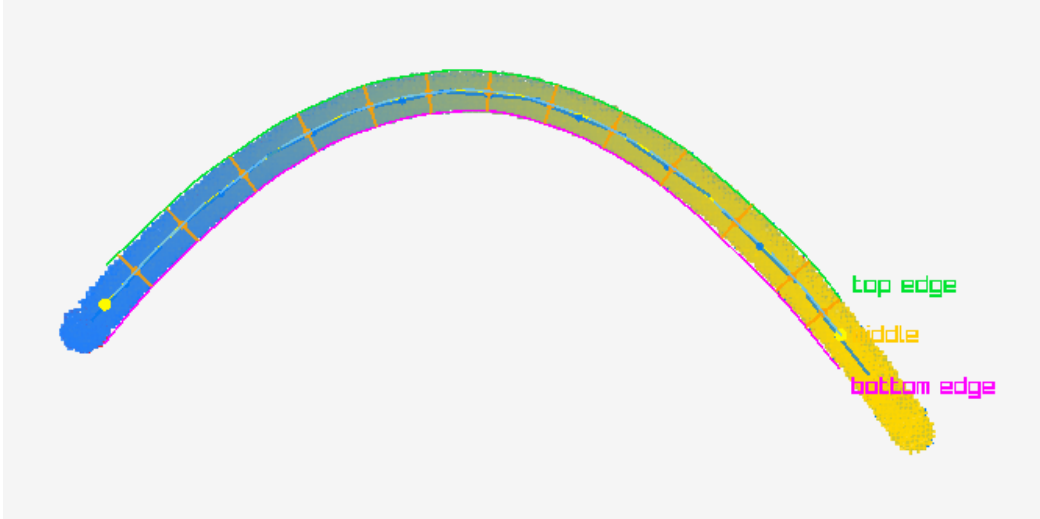


FIGURE 4.8 – Trace test utilisée pour valider la méthode. Les événements forment un ruban courbe : le bord haut est en vert, la ligne médiane en jaune, le bord bas en magenta, et les segments orange représentent les mesures locales de largeur.

4.2.2 Construction des événements de trace

La première étape consiste à construire un nuage d'événements correspondant uniquement à la balle. La méthode utilise pour cela une fenêtre mobile centrée sur la trajectoire 2D déjà estimée. Lorsque la parabole 2D de suivi est disponible, la fenêtre n'est pas un simple disque : elle devient un couloir autour de la courbe prédite. Un événement est conservé s'il se trouve assez près de cette trajectoire, avec une tolérance latérale adaptée au rayon apparent de la balle.

Les événements sont accumulés pendant une courte mémoire temporelle. Cette accumulation est importante : une fenêtre trop courte donne une trace incomplète, alors qu'une mémoire trop longue mélange plusieurs positions et peut épaissir artificiellement le ruban. Le nombre total d'événements accumulés est aussi limité pour éviter une perte de performances. Les polarités peuvent être gardées ensemble ou filtrées séparément, ce qui permet d'isoler le front ou l'arrière de la balle si une polarité est plus stable dans une scène donnée.

4.2.3 Repère local de la trace

Une fois les événements sélectionnés, la méthode cherche le repère naturel du ruban. Le centre c est estimé de manière robuste avec les médianes des coordonnées, puis les points trop éloignés sont retirés à partir d'un quantile radial. Une PCA globale fournit ensuite la direction principale d de la trace. Pour les trajectoires courbes, cette direction est corrigée par des PCA temporelles calculées sur plusieurs sous-parties de la trace. L'orientation finale combine la direction globale, la moyenne pondérée des directions locales et le déplacement temporel entre le début et la fin de la trace.

Le repère local est défini par :

$$s = (p - c) \cdot d, \quad h = (p - c) \cdot n,$$

où p est un événement image et $n = (-d_y, d_x)$ est la normale à la trace. La coordonnée s donne la position le long de la trajectoire, tandis que h donne la position dans l'épaisseur du ruban.

Cette transformation convertit un problème 2D difficile en une série de problèmes 1D : pour chaque petite zone découpée le long de s , il faut trouver les deux bords sur l'axe h .

Il faut distinguer deux notions qui peuvent facilement être confondues. Les sous-parties temporelles utilisées pendant l'estimation de l'orientation servent à stabiliser la direction principale d lorsque la trace est courbe. Elles ne correspondent pas directement aux mesures de largeur. Une fois le repère (s, h) construit, la méthode travaille surtout avec des *bins spatiaux* le long de s . Chaque bin spatial regroupe les événements qui se trouvent dans une petite portion de la trace, puis produit une estimation locale du bord bas, du bord haut et de la ligne médiane. La figure 4.9 montre cette reconstruction dans l'interface.

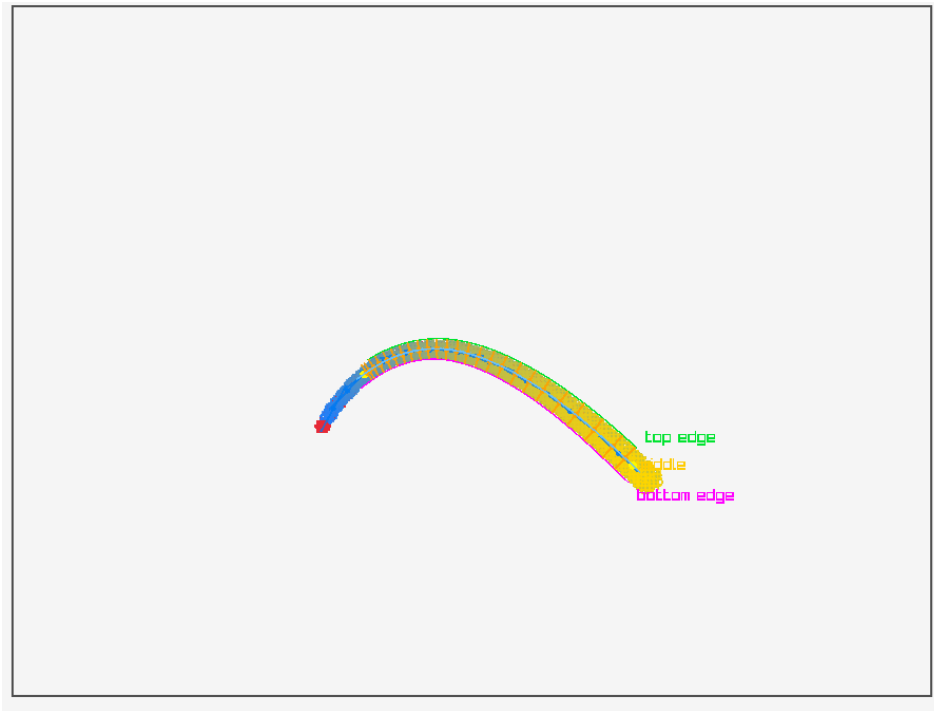


FIGURE 4.9 – Vue Trace dans l'interface pour une balle lancée vers la caméra. Les événements récents sont colorés selon le temps, puis le ruban est reconstruit avec un bord haut, une ligne médiane, un bord bas et plusieurs mesures de largeur.

4.2.4 Découpage spatial et estimation locale de la largeur

La trace n'est pas découpée ici en tranches temporelles, mais en bins spatiaux selon la coordonnée s . Un bin spatial correspond à une petite portion du ruban le long de sa direction principale. Dans chaque bin, on récupère toutes les valeurs h , c'est-à-dire les positions des événements dans l'épaisseur du ruban.

Le point important est le suivant : un bin spatial ne produit pas plusieurs estimations indépendantes de largeur. Il produit une seule estimation locale composée de trois valeurs :

$$(s_j, h_{\text{bas},j}), \quad (s_j, h_{\text{milieu},j}), \quad (s_j, h_{\text{haut},j}).$$

Autrement dit, pour le bin j , l'algorithme cherche où se trouve le bord bas, où se trouve le bord haut, puis il en déduit le milieu :

$$h_{\text{milieu},j} = \frac{h_{\text{haut},j} + h_{\text{bas},j}}{2}.$$

On obtient donc une série d'échantillons le long de la trace :

bin	s_j	$h_{\text{bas},j}$	$h_{\text{milieu},j}$	$h_{\text{haut},j}$
1	s_1	$h_{\text{bas},1}$	$h_{\text{milieu},1}$	$h_{\text{haut},1}$
2	s_2	$h_{\text{bas},2}$	$h_{\text{milieu},2}$	$h_{\text{haut},2}$
3	s_3	$h_{\text{bas},3}$	$h_{\text{milieu},3}$	$h_{\text{haut},3}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

La position s_j d'un bin est généralement prise comme une valeur représentative des coordonnées s des événements du bin, par exemple la médiane. Les valeurs $h_{\text{bas},j}$ et $h_{\text{haut},j}$ sont ensuite estimées à partir de la distribution des valeurs h contenues dans ce bin.

4.2.5 Détection des bords soutenus

La recherche des bords est faite séparément dans chaque bin spatial. La recherche devient donc un problème 1D : à partir de toutes les valeurs h présentes dans le bin, il faut déterminer où commence et où finit le paquet d'événements.

Une solution naïve consisterait à prendre directement le minimum et le maximum des valeurs h :

$$h_{\text{bas}} = \min(h_i), \quad h_{\text{haut}} = \max(h_i).$$

Cette solution est volontairement évitée, car elle serait trop sensible au bruit. Un seul événement isolé très loin du ruban pourrait devenir un faux bord et agrandir artificiellement la largeur. L'algorithme cherche donc des *bords soutenus* : un bord n'est accepté que s'il possède assez de voisins proches sur l'axe h .

Le nombre de voisins requis est calculé à partir du nombre d'événements dans le bin :

$$\text{localSupport} = \text{clamp}\left(\frac{N}{\text{supportDiv}}, \text{supportMin}, \text{supportMax}\right).$$

Les valeurs finies de h sont triées. Pour le bord bas, le balayage part du plus petit h . Pour chaque candidat h_k , on compte combien de valeurs se trouvent dans une fenêtre locale de rayon r_s vers la droite. Le premier candidat qui atteint le support devient le bord bas :

$$h_{\text{bas}} = \min h_k \quad \text{tel que} \quad \#\{h_i \in [h_k, h_k + r_s]\} \geq \text{localSupport}.$$

Pour le bord haut, le principe est symétrique. Le balayage part des grandes valeurs de h , puis vérifie combien de voisins se trouvent dans une fenêtre locale vers la gauche :

$$h_{\text{haut}} = \max h_k \quad \text{tel que} \quad \#\{h_i \in [h_k - r_s, h_k]\} \geq \text{localSupport}.$$

La figure 4.10 illustre cette logique. Les points isolés aux extrémités peuvent avoir une valeur h plus faible ou plus forte que les autres, mais ils ne sont pas retenus comme bords s'ils ne possèdent pas assez de voisins proches.

Recherche des bords par support local sur l'axe h

Les bords ne sont pas les minimums/maximums : ils doivent avoir assez de voisins proches.

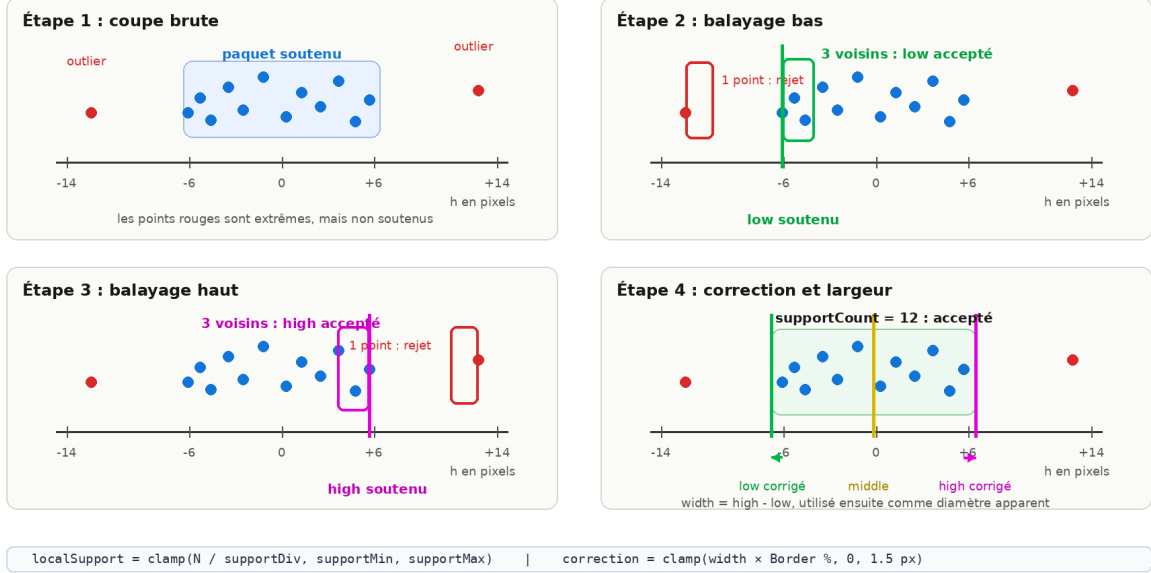


FIGURE 4.10 – Recherche locale des bords par support sur l'axe h . Les événements isolés sont ignorés ; seuls les extrêmes possédant assez de voisins proches deviennent h_{bas} et h_{haut} .

Après la détection des deux extrêmes soutenus, une petite correction extérieure est appliquée. Elle compense le fait que les événements correspondent aux centres des pixels et que le bord apparent peut se trouver légèrement au-delà du dernier événement soutenu. Cette correction est bornée :

$$\delta_b = \text{clamp}((h_{\text{haut}} - h_{\text{bas}}) \text{borderRatio}, 0, 1.5 \text{ px}).$$

Les bords finaux deviennent :

$$h_{\text{bas}} \leftarrow h_{\text{bas}} - \delta_b, \quad h_{\text{haut}} \leftarrow h_{\text{haut}} + \delta_b.$$

La coupe est ensuite validée. Elle est rejetée si elle ne contient pas assez d'événements finis, si $h_{\text{haut}} \leq h_{\text{bas}}$, si la largeur brute est trop faible, ou si le nombre d'événements compris entre les deux bords finaux reste inférieur au support minimal demandé. La largeur locale du bin j est alors :

$$w_j = h_{\text{haut},j} - h_{\text{bas},j}.$$

4.2.6 Filtrage des estimations locales

Après l'étape précédente, chaque bin valide fournit une estimation locale. Cette estimation peut toutefois être mauvaise si un bin contient peu d'événements, si un bruit structuré tombe dans

la coupe, ou si le ruban est localement mal couvert. L'algorithme applique donc un filtrage de cohérence avant de construire les courbes finales du ruban.

Le filtrage se fait en deux niveaux. Le premier niveau vérifie la cohérence globale du ruban. À partir des lignes médianes détectées dans les bins, un modèle polynomial simple est ajusté, linéaire ou quadratique selon le nombre d'échantillons disponibles. Ce modèle ne sert pas encore à tracer définitivement le ruban. Il sert d'abord à repérer les bins incohérents : un bin est rejeté si sa ligne médiane est trop éloignée de la tendance générale ou si sa largeur locale est trop différente de la largeur médiane du ruban.

Le second niveau vérifie la cohérence locale. Pour chaque bin j , on regarde les bins voisins et prédit localement la valeur attendue du bord haut, du bord bas, de la ligne médiane et de la largeur. L'idée est simple : si les bins voisins disent que le ruban passe à un certain endroit avec une certaine largeur, le bin courant ne doit pas créer un saut brutal sans raison.

On peut résumer cette étape par une prédiction locale :

$$\hat{h}_{\text{bas},j}, \quad \hat{h}_{\text{milieu},j}, \quad \hat{h}_{\text{haut},j}, \quad \hat{w}_j.$$

La prédiction est construite à partir d'une petite régression locale sur les échantillons voisins. Ce ne sont donc pas seulement les voisins immédiats qui comptent. Selon le paramétrage, cette fenêtre peut couvrir plusieurs bins autour de j . Par exemple, le bin j peut être comparé aux bins $j - 4$ à $j + 4$, en ignorant le bin courant pour éviter qu'il ne se valide lui-même.

Un bin est considéré comme aberrant s'il casse la continuité locale. Pour la largeur, le principe peut s'écrire :

$$|w_j - \hat{w}_j| > \tau_w.$$

La même logique est appliquée au bord bas, à la ligne médiane et au bord haut :

$$|h_{\text{bas},j} - \hat{h}_{\text{bas},j}| > \tau_{\text{bas}}, \quad |h_{\text{milieu},j} - \hat{h}_{\text{milieu},j}| > \tau_{\text{milieu}}, \quad |h_{\text{haut},j} - \hat{h}_{\text{haut},j}| > \tau_{\text{haut}}.$$

Les seuils dépendent de la largeur médiane du ruban et de la dispersion locale, souvent estimée par une MAD. L'intérêt de cette étape est d'éviter deux erreurs opposées. Premièrement, elle évite de garder un bin totalement incohérent qui ferait exploser la largeur et donc fausserait la profondeur. Deuxièmement, elle évite de supprimer un bin simplement parce qu'il n'est pas exactement égal à la moyenne globale : il est supprimé seulement s'il est incohérent avec son voisinage. La figure 4.11 résume ce filtrage local.

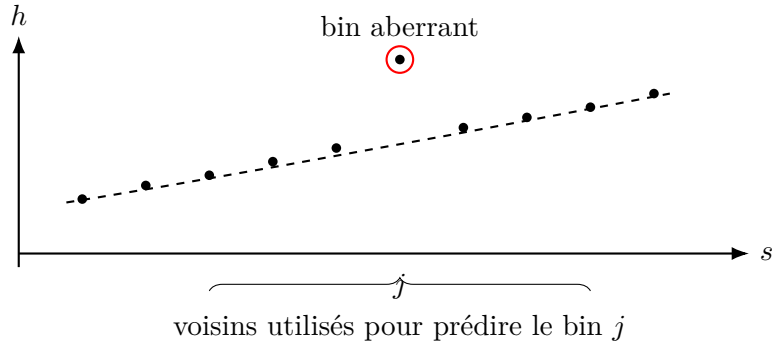


FIGURE 4.11 – Principe du filtrage local. Le bin courant est comparé à une prédiction construite avec ses voisins ; s’il crée une rupture trop forte, il est supprimé.

4.2.7 Courbes du ruban

Après le filtrage, il reste trois listes d’échantillons propres :

$$\mathcal{B} = \{(s_j, h_{\text{bas},j})\}, \quad \mathcal{M} = \{(s_j, h_{\text{milieu},j})\}, \quad \mathcal{H} = \{(s_j, h_{\text{haut},j})\}.$$

Ces points ne sont pas reliés par une régression séparée dans chaque bin. Un bin ne contient qu’une mesure locale. Les courbes sont construites après coup, à partir de l’ensemble des bins valides restants. Il ne s’agit pas non plus nécessairement d’une seule grande parabole globale imposée sur toute la trace. L’algorithme utilise un ajustement local : autour de la position s_0 , les échantillons proches de s_0 ont plus de poids que les échantillons éloignés.

La bonne lecture est donc la suivante :

- les bins servent à produire les points de mesure (s_j, h_j) ;
- le filtrage supprime les points incohérents ;
- les courbes haut, milieu et bas sont ajustées sur tous les échantillons restants, mais avec un poids local autour de la position évaluée.

Pour un ajustement local quadratique, la forme utilisée autour de s_0 est :

$$h(s) = a(s - s_0)^2 + b(s - s_0) + c.$$

Le fait de centrer le modèle sur s_0 est pratique : la valeur estimée au point courant est directement c , puisque $s = s_0$ donne :

$$h(s_0) = c.$$

Les coefficients sont recalculés localement pour plusieurs positions s_0 le long du ruban. L’idée générale est de donner davantage d’influence aux échantillons proches de la position évaluée et moins d’influence aux échantillons éloignés. Ainsi, la courbe reste régulière sans être forcée par une seule régression globale sur toute la trace. Si une zone contient peu d’échantillons, la fenêtre locale peut être élargie afin de conserver une estimation stable.

Les trois courbes sont ajustées indépendamment :

$$h_{\text{bas}}(s), \quad h_{\text{milieu}}(s), \quad h_{\text{haut}}(s).$$

Cela permet au ruban de rester souple. Le bord haut et le bord bas ne sont pas forcés à avoir exactement la même forme, ce qui est utile si la trace est légèrement asymétrique, si une polarité domine localement, ou si une zone contient moins d'événements.

Pour tracer une courbe complète, plusieurs positions s_0 sont échantillonnées entre le début et la fin du ruban. À chaque position, un petit ajustement local est recalculé afin de récupérer la valeur estimée. Le point est ensuite reconverti dans le repère image avec :

$$p(s_0) = c_{\text{trace}} + s_0 d + h(s_0) n,$$

où c_{trace} est le centre du ruban, d sa direction principale et n sa normale. Cette opération est faite pour la ligne médiane, le bord bas et le bord haut. La figure 4.12 illustre l'ajustement local des trois courbes du ruban.

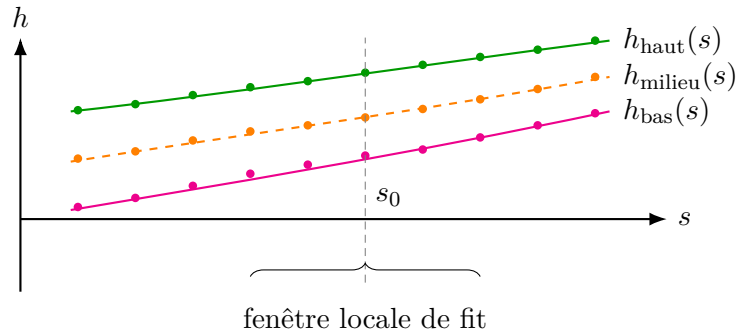


FIGURE 4.12 – Les trois courbes du ruban sont ajustées à partir des échantillons valides. L'ajustement est local : autour de s_0 , les points proches pèsent davantage que les points éloignés.

4.2.8 Conversion 3D par largeur de trace

La conversion 3D repose sur le même principe géométrique que le circle fitting, mais la grandeur mesurée n'est plus le rayon d'un cercle. Pour plusieurs instants de la trace, la méthode mesure une largeur perpendiculaire à la ligne médiane. Cette largeur est utilisée comme diamètre apparent de la balle :

$$Z = \frac{f_{\text{eff}} D}{w_{px}},$$

où D est le diamètre réel de la balle et w_{px} la largeur mesurée en pixels. Comme la largeur n'est pas forcément horizontale ou verticale dans l'image, la méthode utilise une focale effective dans la direction de mesure :

$$f_{\text{eff}} = \sqrt{(f_x n_x)^2 + (f_y n_y)^2}.$$

Le centre image (u, v) de la mesure permet ensuite de retrouver les coordonnées caméra :

$$X = \frac{(u - c_x)Z}{f_x}, \quad Y = \frac{(v - c_y)Z}{f_y}.$$

Dans l’affichage 3D, ces coordonnées sont ensuite remappées dans le repère utilisé par l’interface. Les positions 3D aberrantes sont filtrées par sauts temporels et par résidu de trajectoire. Le résultat final est donc une suite de points 3D, puis une trajectoire ajustée sur ces points. Les sorties obtenues avec cette conversion sont présentées dans la section 4.3, où la simulation fournit une vérité terrain pour comparer directement l’estimation à la position réelle de la balle.

4.2.9 Régression pondérée de trajectoire

Une régression est ensuite calculée sur les positions 3D obtenues. Le modèle reste volontairement simple, comme pour la première méthode :

$$x(t) = at + b, \quad y(t) = at + b, \quad z(t) = at^2 + bt + c.$$

Les axes x et y sont modélisés par une droite car la vitesse horizontale varie peu sur la courte durée observée. L’axe z est modélisé par un polynôme du second degré pour représenter la courbure verticale de la trajectoire. La différence entre la version non pondérée et la version pondérée vient du poids associé à chaque observation.

La pondération combine deux termes. Le premier donne plus d’importance aux points récents :

$$w_{\text{temps},i} = \exp(-3 \text{age}_i).$$

Le second réduit l’influence des points dont le résidu est élevé :

$$w_{\text{robuste},i} = \frac{1}{1 + \left(\frac{r_i}{\sigma}\right)^2}.$$

Le poids final est donc :

$$w_i = w_{\text{temps},i} w_{\text{robuste},i}.$$

Sans pondération, une mauvaise estimation ancienne peut tirer la courbe finale et créer un décalage visible. Avec pondération, la trajectoire suit davantage les points récents tout en diminuant l’effet des mesures aberrantes. Cela ne change pas le modèle physique, mais rend l’estimation plus stable pour une utilisation en temps réel. La comparaison visuelle entre la version pondérée et la version non pondérée est donc placée dans la section de validation, afin de garder ici l’explication de l’algorithme et de réserver les résultats expérimentaux à la suite de la partie perception.

4.2.10 Bilan de la méthode Trace

La méthode Trace est plus prometteuse que le circle fitting pour les tirs rapides, car elle exploite la forme réellement produite par la balle dans le flux événementiel. Au lieu de dépendre fortement d’un rayon instantané, elle mesure une largeur sur plusieurs coupes de la trace et combine ces mesures dans le temps. Elle reste sensible aux erreurs de largeur, à la qualité de la fenêtre de suivi et à la quantité d’événements disponibles, mais son information de départ est mieux adaptée au signal observé par la caméra événementielle.

4.3 Simulation et validation

4.3.1 Séquences simulées

Pour tester l'algorithme dans un cadre contrôlé, des séquences simulées ont été générées avec Isaac Sim [9]. Une balle est lancée devant une caméra virtuelle dont les paramètres intrinsèques sont les mêmes que ceux utilisés par l'algorithme. La sortie vidéo de cette caméra est ensuite convertie en événements avec **v2e** [11, 12], afin d'obtenir un flux proche de celui fourni par une caméra événementielle. En parallèle, Isaac Sim [9] fournit la position réelle de la balle à chaque instant.

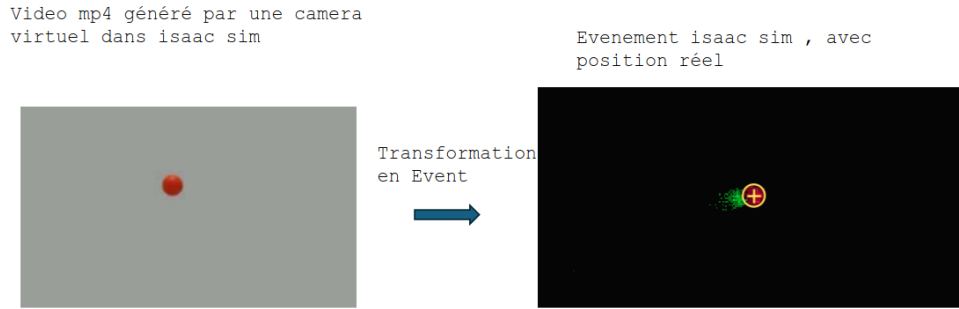


FIGURE 4.13 – Passage d'une vidéo générée en simulation vers un flux événementiel avec **v2e** [11, 12].

Cette organisation permet de comparer directement les positions estimées par l'algorithme avec une vérité terrain. Dans les figures 3D de cette section, chaque carré de la grille représente 25 cm. Les estimations de l'algorithme sont affichées en bleu, tandis que les positions réelles de référence sont affichées en rouge.

Les essais simulés ont été réalisés avec les paramètres suivants :

- résolution vidéo : 640×480 ;
- fréquence d'entrée : 500 fps ;
- rayon de balle : 0.02 m ;
- calibration synthétique : $f_x = f_y = 520$ px ;
- résolution temporelle des événements : $200 \mu s$;
- seuils de génération ON/OFF : 0.15.

TABLE 4.2 – Caractéristiques des séquences simulées utilisées pour la validation.

Séquence	Durée	Événements bruts	Événements filtrés
sequence_0001	2.0 s	49 232	38 981
sequence_0002	2.0 s	47 835	37 909
sequence_0003	0.468 s	28 016	22 478

La vérité terrain contient la position réelle de la balle dans le repère caméra, ainsi que sa projection image. Elle permet de comparer la position estimée par l'algorithme avec la trajectoire attendue. Cette comparaison est importante, car une trajectoire peut sembler visuellement co-

hérente tout en ayant une erreur de profondeur trop élevée pour l'interception robotique.

4.3.2 Résultats 3D de la méthode Trace

La figure 4.14 montre les sorties 3D obtenues avec la méthode Trace sur une séquence simulée. Les points bleus clairs correspondent aux mesures de largeur converties en positions 3D. La courbe bleue est la trajectoire estimée à partir de ces points. Les points rouges représentent la trajectoire réelle fournie par Isaac Sim [9].

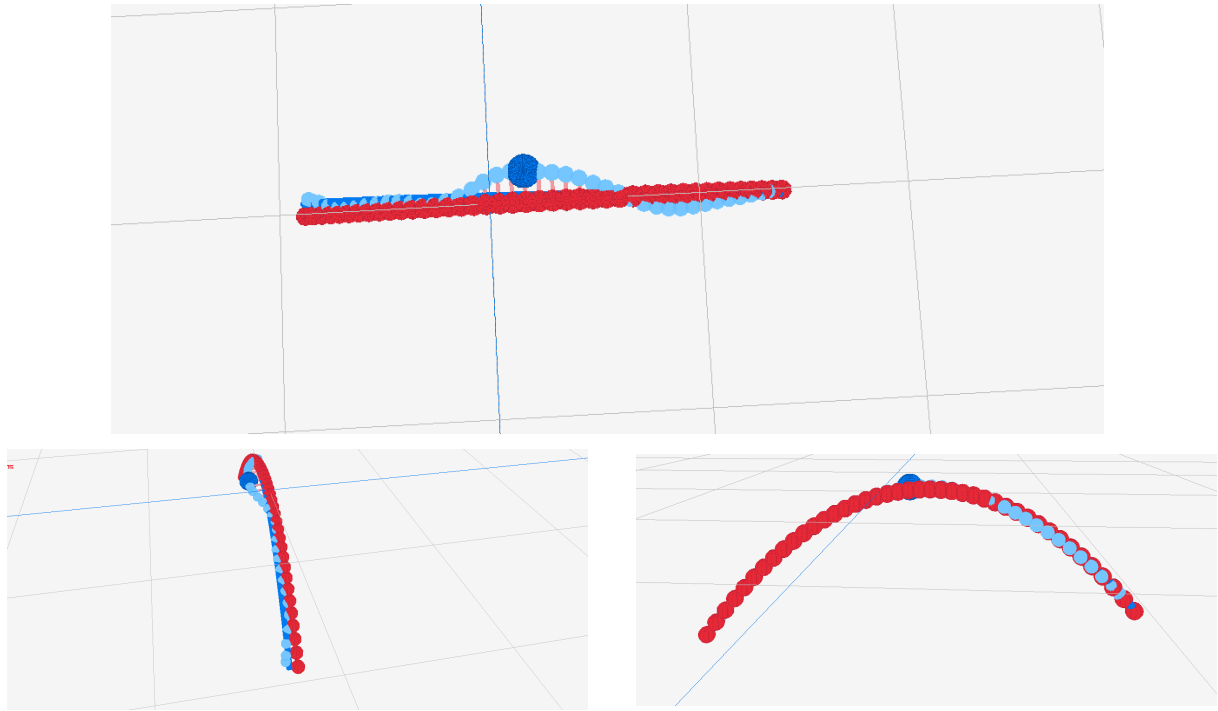


FIGURE 4.14 – Sorties 3D obtenues avec la méthode Trace. Chaque carré de la grille correspond à 25 cm. Les points bleus sont les estimations de l'algorithme et les points rouges sont les positions réelles issues d'Isaac Sim [9].

La figure 4.15 détaille l'évolution de cette trajectoire lorsque de nouvelles estimations sont ajoutées au fil du temps.

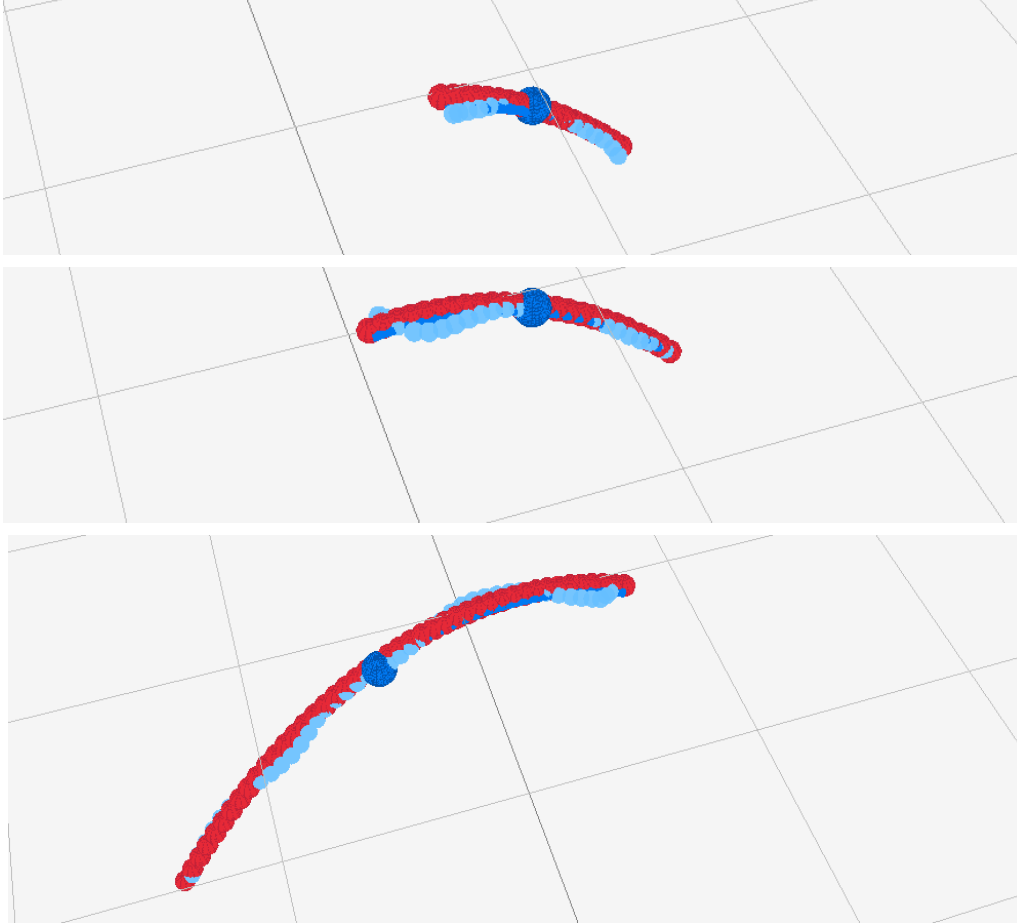


FIGURE 4.15 – Évolution de la trajectoire Trace dans le temps. À mesure que de nouvelles largeurs sont mesurées, de nouveaux points 3D sont ajoutés et la courbe estimée se stabilise.

4.3.3 Effet de la régression pondérée

La régression pondérée est évaluée sur la même sortie 3D. Sans poids ajustés, la trajectoire bleue peut être attirée par des points anciens ou par des estimations moins fiables. Avec les poids de récence et de robustesse, la courbe tient davantage compte des points récents et réduit l'influence des mesures aberrantes. L'objectif n'est pas de changer le modèle de trajectoire, mais de rendre son estimation plus exploitable en temps réel. La figure 4.16 compare visuellement ces deux réglages.

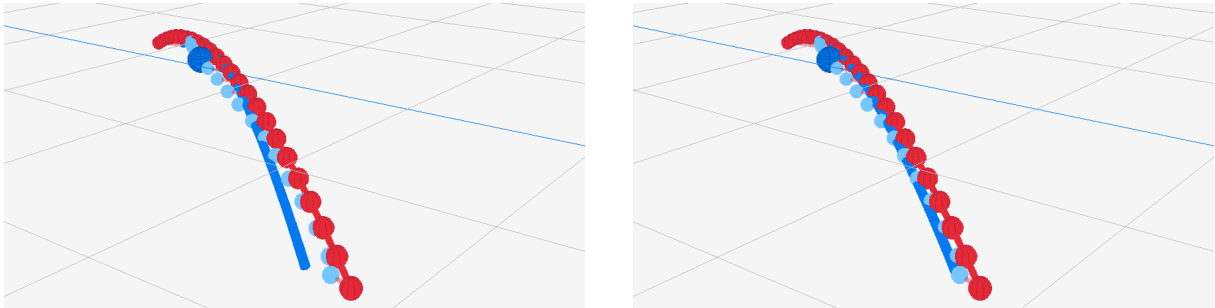


FIGURE 4.16 – Comparaison entre la régression Trace sans poids ajustés à gauche et avec poids ajustés à droite, pour une balle lancée vers la caméra. La pondération récente et robuste aligne mieux la courbe bleue sur les estimations utiles de la trajectoire.

4.3.4 Rôle des tests

Les séquences simulées servent à séparer deux problèmes :

- la qualité de l'algorithme géométrique ;
- les difficultés liées au matériel réel, à l'éclairage et au bruit de la caméra.

Lorsque l'algorithme échoue sur une séquence simulée avec vérité terrain, l'erreur vient principalement du traitement ou du modèle géométrique. Lorsqu'il fonctionne en simulation mais devient instable sur la caméra réelle, il faut plutôt analyser la calibration, la polarité des événements, la densité de la trace et les conditions expérimentales.

Chapitre 5

Intégration robotique et contrôle

5.1 Calibration extrinsèque par mire événementielle

5.1.1 Intuition générale

Les méthodes de perception précédentes estiment la balle dans le repère de la caméra. Pour qu'un robot puisse utiliser cette information, il faut exprimer cette position dans le repère de sa base. Le problème de calibration extrinsèque consiste donc à estimer une transformation fixe entre la caméra événementielle et la base du robot :

$${}^B T_C.$$

Le principe retenu est d'utiliser le robot lui-même comme système de déplacement connu. Une mire dynamique est affichée sur un écran fixé au bout du bras robotique. Comme l'écran est solidaire de l'effecteur, sa pose dans la base du robot peut être calculée à partir de la cinématique directe et de la géométrie du support mécanique. La caméra événementielle observe cette mire pour plusieurs poses du robot. Chaque observation donne une contrainte géométrique, et l'ensemble des poses permet d'estimer la transformation fixe entre la caméra et le robot. La figure 5.1 présente une première idée de mire dynamique.

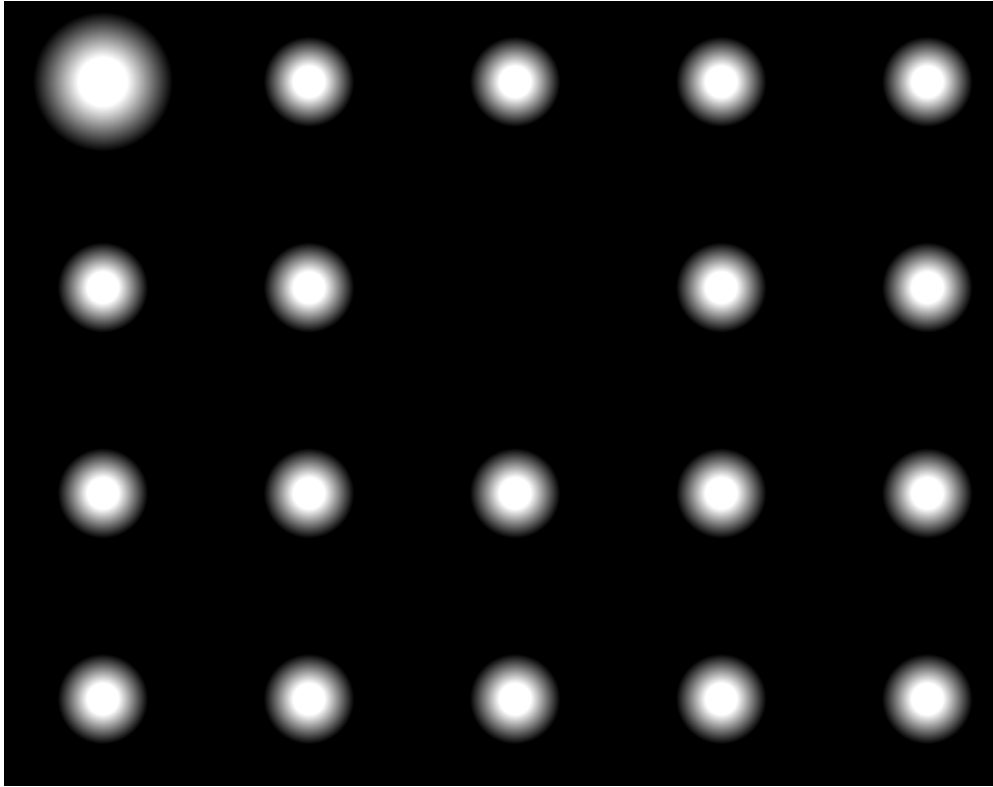


FIGURE 5.1 – Mire conceptuelle affichée sur un écran. Les disques lumineux peuvent clignoter pour générer une activité événementielle exploitable par la caméra DVXplorer [4].

Cette approche est intéressante car elle ne demande pas de déplacer manuellement une mire dans l'espace. Le robot fournit directement des poses connues et répétables, ce qui rend possible une calibration multi-poses cohérente.

5.1.2 Montage envisagé

Le montage repose sur quatre éléments principaux :

- une caméra événementielle DVXplorer [4] fixe par rapport à l'environnement ;
- un robot 6 DOF dont la cinématique directe fournit la pose de l'effecteur ;
- un support imprimé en 3D reliant l'effecteur à un écran ;
- une mire lumineuse affichée sur cet écran.

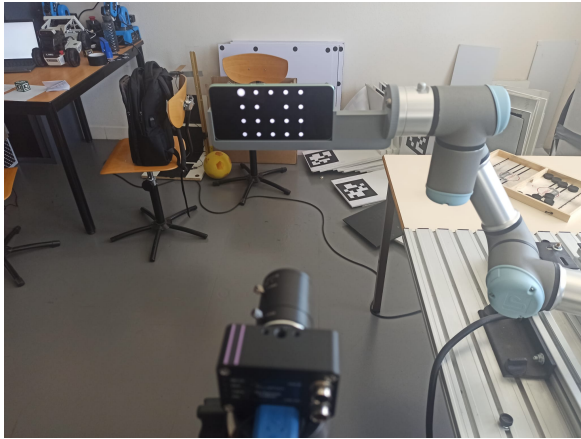


FIGURE 5.2 – Smartphone installé sur le support utilisé pour afficher la mire dynamique. À gauche, vue de face du montage ; à droite, vue de côté montrant le placement du téléphone sur le support.

L'écran peut être un smartphone, une tablette légère ou un petit écran dédié. Un écran trop lourd est à éviter, car il peut provoquer une flexion du support ou de l'effecteur et fausser la pose réelle de la mire. La rigidité mécanique est donc aussi importante que la précision algorithmique : si la transformation entre l'effecteur et l'écran varie pendant le mouvement, la calibration cherchera à expliquer une erreur qui ne vient pas de la caméra.

5.1.3 Repères et transformations

On utilise les repères suivants :

- B : repère de la base du robot ;
- E : repère de l'effecteur, de la flange ou du TCP robot ;
- S : repère de la mire affichée sur l'écran ;
- C : repère de la caméra événementielle.

Pour une pose robot i , la cinématique directe donne la transformation ${}^B T_{E,i}$. La transformation ${}^E T_S$ décrit le placement mécanique de la mire par rapport à l'effecteur. Dans le montage envisagé, elle provient uniquement de la CAO du support imprimé en 3D. Le support est conçu de telle sorte que le smartphone soit maintenu sans jeu et que sa position par rapport à l'effecteur reste fixe pendant les acquisitions.

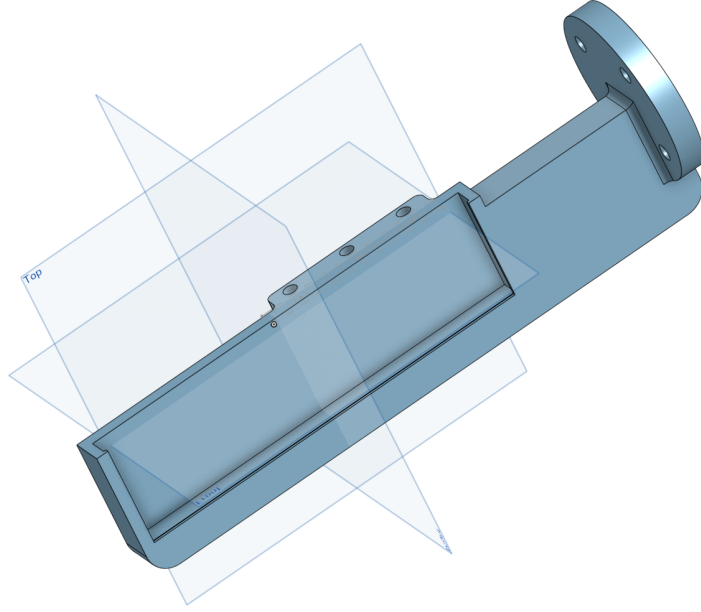


FIGURE 5.3 – Modèle CAO du support utilisé pour définir la position de la mire par rapport à l’effecteur.

La position réelle de l’écran dans le téléphone doit aussi être prise en compte, car la mire n’occupe pas toute la surface physique du smartphone. Cette position est estimée à partir d’une image du téléphone : on mesure en pixels l’épaisseur des bords entre le contour du téléphone et la zone active de l’écran, puis on convertit ces valeurs en millimètres par proportionnalité, en utilisant les dimensions réelles du téléphone comme référence. Cette méthode revient à appliquer une règle de trois entre les dimensions mesurées sur l’image et les dimensions physiques.

TABLE 5.1 – Estimation des bords entre le téléphone et la zone active de l’écran.

Élément	En pixels	En millimètres estimés
Bord gauche	≈ 23 px	$\approx 2,92$ mm
Bord droit	≈ 21 à 22 px	$\approx 2,67$ à $2,80$ mm
Bord haut	≈ 23 px	$\approx 2,93$ mm
Bord bas	≈ 26 px	$\approx 3,31$ mm

La pose de la mire dans la base du robot est alors :

$${}^B T_{S,i} = {}^B T_{E,i} {}^E T_S.$$

De son côté, la caméra observe la mire et permet d’estimer la transformation ${}^C T_{S,i}$, c’est-à-dire la pose de la mire dans le repère caméra. Pour une pose isolée, on peut en déduire une première estimation de la pose caméra dans la base :

$${}^B T_{C,i} = {}^B T_{S,i} \left({}^C T_{S,i} \right)^{-1}.$$

Une seule pose reste cependant fragile : une erreur de détection, un mauvais angle de vue ou une légère erreur mécanique peut produire une estimation biaisée. L’objectif réel est donc d’utiliser

plusieurs poses et de trouver une transformation unique ${}^B T_C$ qui explique au mieux toutes les observations.

5.1.4 Design de la mire dynamique

La mire envisagée est composée de disques lumineux sur fond noir. Les disques alternent entre un état visible et un état invisible, ou entre deux niveaux de luminosité, afin de générer des événements dans la caméra. Pour une caméra événementielle, l'intérêt n'est pas seulement la forme visible à l'oeil, mais surtout la variation temporelle de luminosité produite par la mire.

Ce choix est directement lié à la méthode utilisée dans le script Python. Celui-ci ne cherche pas des coins comme avec un échiquier classique : il construit une image d'activité, applique un léger flou, réalise un seuillage, puis regroupe les pixels actifs en composantes connexes. Chaque composante est alors considérée comme un *blob*. Le centre de chaque blob est ensuite estimé par barycentre pondéré de l'activité. La mire doit donc produire des zones actives compactes, séparées et faciles à associer à des points physiques connus.

Les disques sont adaptés à cette détection par blobs, car leur projection donne une composante compacte avec un centre bien défini. Un gradient radial peut aussi être utilisé : le centre du disque reste plus lumineux, tandis que les bords deviennent plus progressifs. Cette forme peut rendre l'activité moins brutale et aider le barycentre à être moins dépendant d'un pixel isolé. En revanche, un gradient trop doux peut rendre le seuil de détection moins net, car les bords du blob deviennent plus diffus. Le réglage du gradient doit donc rester un compromis entre stabilité du centre et séparation claire des blobs.

La mire ne doit pas être parfaitement ambiguë. Une grille régulière de disques identiques peut être difficile à orienter si plusieurs associations 2D/3D sont possibles. Le design retenu utilise donc une grille asymétrique, avec un disque manquant et un disque plus grand servant d'ancre. Le disque plus grand produit généralement un blob plus fort, ce qui aide le script à identifier l'orientation de la mire et à associer les blobs détectés aux points attendus.

La figure 5.1 illustre cette idée de mire : un fond noir, des disques lumineux, un disque manquant et un disque plus grand permettant d'aider l'identification de l'orientation.

5.1.5 Extraction des points dans les événements

La caméra événementielle fournit des événements de la forme :

$$e_k = (u_k, v_k, t_k, p_k),$$

où (u_k, v_k) sont les coordonnées image, t_k le timestamp et p_k la polarité. Pour détecter une mire clignotante, l'idée générale est d'accumuler l'activité sur une courte fenêtre temporelle autour du changement d'état de la mire. Dans cette première approche, les polarités positives et négatives ne sont pas soustraites : elles sont toutes les deux considérées comme de l'activité utile.

On peut donc construire une image d'activité :

$$A(u, v) = \sum_{k \in \mathcal{W}} \mathbf{1}[(u_k, v_k) = (u, v)],$$

avec $\mathcal{W}$ la fenêtre temporelle utilisée. Les zones où l'activité est forte correspondent aux disques

qui viennent de changer de luminosité. Après seuillage et regroupement spatial, chaque composante détectée peut être associée à un disque de la mire.

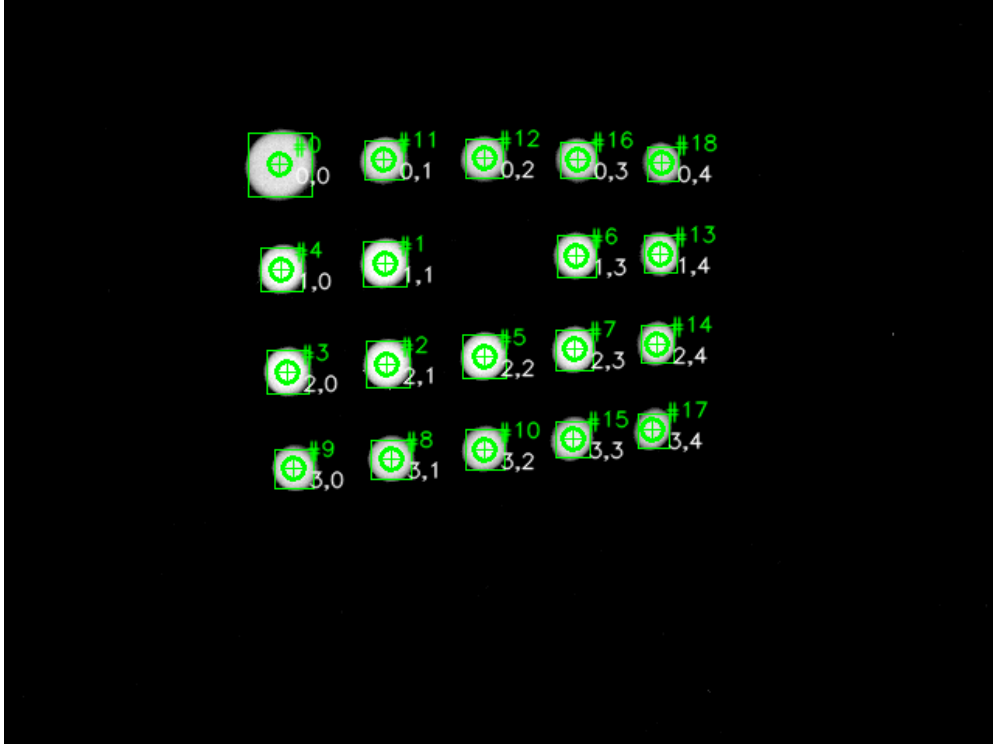


FIGURE 5.4 – Exemple de sortie de l’algorithme de détection de la mire événementielle. Les blobs détectés sont associés aux points physiques de la mire, puis annotés avec leur identifiant et leurs coordonnées dans la grille.

Le centre d’un disque peut être estimé par barycentre pondéré :

$$u_c = \frac{\sum u A(u, v)}{\sum A(u, v)}, \quad v_c = \frac{\sum v A(u, v)}{\sum A(u, v)}.$$

Ce centre est généralement plus stable qu’un simple centre de boîte englobante, car il utilise la distribution complète de l’activité dans le blob. Si la mire est vue avec un angle important, un ajustement d’ellipse pourrait aussi être envisagé, mais le barycentre constitue une première estimation simple et rapide.

5.1.6 Association 2D/3D et pose de la mire

Pour estimer la pose de la mire dans la caméra, il faut associer les centres détectés dans l’image aux points physiques connus de la mire. Les coordonnées des points dans le repère S sont coplanaires, car ils appartiennent à la surface de l’écran :

$$P_{ij}^S = \begin{bmatrix} X_{ij} \\ Y_{ij} \\ 0 \end{bmatrix}.$$

Pour une grille de cols colonnes et rows lignes, avec des espacements s_x et s_y , on peut définir :

$$X_{ij} = \left(i - \frac{\text{cols} - 1}{2}\right) s_x, \quad Y_{ij} = \left(j - \frac{\text{rows} - 1}{2}\right) s_y.$$

Ces points 3D sont mis en correspondance avec les centres image (u_{ij}, v_{ij}) . La pose ${}^C T_{S,i}$ peut alors être estimée avec une méthode de type solvePnP [7], en utilisant les paramètres intrinsèques de la caméra événementielle et, si nécessaire, les coefficients de distorsion. Une bonne calibration intrinsèque est donc un prérequis : une erreur dans les paramètres caméra se propagera directement dans la transformation extrinsèque.

5.1.7 Calibration multi-poses

La solution pose par pose permet de vérifier que le système fonctionne, mais elle ne constitue pas la méthode finale. La calibration doit plutôt chercher une transformation unique qui minimise l'erreur de reprojection sur toutes les poses robot et tous les points observés.

Pour chaque pose i et chaque point j , on connaît :

- la position du point P_j^S dans la mire ;
- la pose robot de la mire ${}^B T_{S,i}$;
- l'observation image $\hat{\mathbf{u}}_{ij}$;
- les paramètres intrinsèques de la caméra.

En cherchant ${}^C T_B$, l'erreur de reprojection globale peut s'écrire :

$$\min_{{}^C T_B} \sum_{i,j} \left\| \pi \left({}^C T_B {}^B T_{S,i} P_j^S \right) - \hat{\mathbf{u}}_{ij} \right\|^2,$$

où π représente la projection caméra. Cette formulation est plus robuste qu'une moyenne directe des estimations ${}^B T_{C,i}$, car elle ajuste la transformation finale à partir des erreurs réellement visibles dans l'image.

5.1.8 Paramètres physiques de l'écran

La mire affichée sur l'écran doit être connue en dimensions réelles. Dans l'outil Python développé, cette étape est automatisée : le programme récupère la résolution de l'écran ainsi que sa taille physique, puis en déduit directement le facteur de conversion entre pixels et millimètres. Aucune mesure manuelle de l'écran n'est donc nécessaire pour générer la mire avec les bonnes dimensions. La densité en pixels par millimètre est calculée par :

$$\text{px/mm}_x = \frac{\text{largeur en pixels}}{\text{largeur en mm}}, \quad \text{px/mm}_y = \frac{\text{hauteur en pixels}}{\text{hauteur en mm}}.$$

Le script utilise ces valeurs pour définir automatiquement le diamètre des disques, leur espacement et l'origine du repère S . Cela évite d'introduire une erreur supplémentaire liée à une mesure faite à la main, tout en gardant une correspondance directe entre les coordonnées pixels de la mire et ses coordonnées physiques en millimètres.

5.1.9 Tests de précision et points critiques

Des tests ont été réalisés avec l'outil Python afin de vérifier que la pose estimée à partir de la mire permet bien de prédire d'autres points affichés sur le même écran. Le test se déroule en deux phases. Dans un premier temps, la mire complète à 19 points est affichée et détectée, puis une pose de l'écran est calculée avec `solvePnP` [7]. Dans un second temps, le programme affiche une série de cibles carrées à quatre points, avec des positions et des tailles différentes. Les quatre coins attendus sont projetés dans l'image à partir de la pose estimée, puis comparés aux blobs réellement détectés.

Deux erreurs différentes sont alors calculées. La première est l'erreur de reprojection en pixels : chaque coin réel du carré est projeté dans l'image, puis comparé au centre du blob détecté correspondant. Le score affiché est un RMS sur ces distances image :

$$e_{\text{repr}} = \sqrt{\frac{1}{N} \sum_i \|\hat{\mathbf{u}}_i - \mathbf{u}_i\|^2}.$$

La seconde erreur est calculée en millimètres. Les quatre coins détectés sont reprojetés sur le plan estimé de l'écran, ce qui permet de reconstruire les quatre longueurs du carré. Ces longueurs mesurées sont ensuite comparées aux longueurs réelles connues :

$$e_{\text{cote}} = \frac{1}{4} \sum_{k=1}^4 \left| L_k^{\text{mesure}} - L_k^{\text{reel}} \right|.$$

La figure 5.5 montre une sortie de ce test. Sur cet essai, les quatre carrés de validation sont acceptés. L'erreur de reprojection globale est d'environ 0,13 px, tandis que l'erreur moyenne sur les longueurs reconstruites des côtés est d'environ 0,16 mm. La pose utilisée pour cette validation place l'écran à environ 634 mm de la caméra, avec une inclinaison d'environ 20°. Ces valeurs ne constituent pas encore une précision finale de calibration robot-caméra, mais elles montrent que la chaîne de détection, d'association, de projection et de reconstruction métrique est cohérente sur le plan de l'écran.



FIGURE 5.5 – Sortie du test carré utilisé pour valider la cohérence géométrique de la mire événementielle. Les carrés projetés à partir de la pose estimée sont comparés aux blobs détectés dans les événements.

La précision finale de la calibration extrinsèque dépendra ensuite à la fois de cette détection événementielle, de la géométrie du montage mécanique et de la diversité des poses robot utilisées. L'erreur angulaire reste particulièrement importante : une erreur d'orientation de 1° représente environ 17 mm d'erreur latérale à 1 m, et environ 35 mm à 2 m. Pour une tâche d'interception de balle, cette erreur peut devenir aussi critique qu'une erreur directe sur la profondeur.

Le protocole expérimental devra donc privilégier :

- des poses nombreuses et variées, avec des orientations différentes de la mire ;
- une couverture large de l'image caméra ;
- des plans de mire non tous parallèles ;
- une correction correcte de la distorsion ;
- le rejet des poses où les blobs sont mal détectés ;
- l'analyse des résidus de reprojection après optimisation.

5.1.10 Lien avec l'interception robotique

Cette calibration est l'étape qui rend la perception exploitable par le robot. Une fois ${}^B T_C$ connue, une position de balle estimée dans le repère caméra peut être transformée dans le repère de la base :

$$P^B = {}^B T_C P^C.$$

La trajectoire obtenue par la méthode Trace peut alors être exprimée dans le repère robot, filtrée, prédite, puis transmise à la commande ou à une politique d'interception. La qualité de cette transformation conditionne directement la précision finale : même une bonne estimation

caméra ne suffit pas si la calibration extrinsèque déplace la balle de plusieurs centimètres dans le repère du robot.

5.2 Contrôle robotique : Isaac Lab et PPO

5.2.1 Objectif de la partie robot

La perception seule ne suffit pas pour attraper la balle. Elle doit être intégrée à une commande robotique capable de déplacer l'effecteur vers une position d'interception. Cette partie a été étudiée dans Isaac Lab [10] avec un apprentissage par renforcement de type PPO [13], pour *Proximal Policy Optimization*.

L'objectif de PPO [13] est d'apprendre une politique de commande. À partir de l'état courant du robot et de la balle, la politique choisit une action qui rapproche le robot d'une situation favorable pour intercepter la balle. La simulation permet de répéter un grand nombre d'essais sans risque pour le matériel réel. La figure 5.6 montre le montage réel considéré pour relier la perception à la robotique.

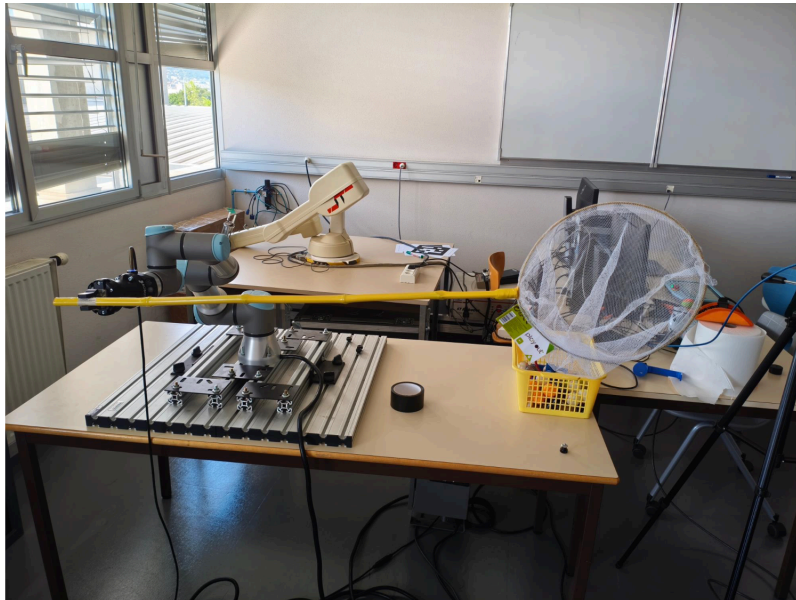


FIGURE 5.6 – Montage réel avec robot, caméra et zone de réception.

5.2.2 Résultats d'entraînement observés

D'après les essais présentés pendant le stage, l'entraînement atteint une convergence autour de 15 000 itérations. L'entraînement complet a été poussé jusqu'à environ 35 000 itérations. Avec 32 rollouts et environ 12 000 environnements, cela représente approximativement :

$$15\,000 \times 32 \times 12\,000 = 5.76 \times 10^9$$

steps de simulation pour la phase de convergence. Le temps total observé est d'environ 26 minutes, avec environ 11 minutes pour atteindre la convergence.

Ces chiffres montrent que la simulation permet d'entraîner rapidement un comportement de base. Cependant, le passage vers le système réel demande de prendre en compte plusieurs défis supplémentaires.

5.2.3 Défis pour l'intégration perception-robot

TABLE 5.2 – Principaux défis pour passer de la perception à l'interception robotique.

Défi	Impact attendu
Latence perception-commande	Une estimation correcte mais trop tardive peut déplacer le point d'interception. Il faut mesurer le temps entre acquisition, traitement, publication ROS 2 [6] et action robot.
Synchronisation temporelle	La trajectoire doit être associée à des timestamps fiables. Sans synchronisation, le robot peut suivre une balle déjà passée.
Calibration caméra-robot	La position 3D estimée dans le repère caméra doit être transformée dans le repère du robot avec une extrinsèque précise.
Espace atteignable	Toutes les trajectoires de balle ne sont pas interceptables. Il faut vérifier la zone de travail, les limites articulaires et la vitesse maximale du robot.
Bruit de mesure	Une erreur de quelques centimètres en profondeur peut suffire à rater la balle. La méthode Trace réduit ce risque, mais un filtrage temporel reste nécessaire.
Sim-to-real	Une politique apprise en simulation peut échouer si la dynamique réelle, les délais ou les observations diffèrent trop du simulateur.
Sécurité	Les essais réels doivent limiter les vitesses, gérer les arrêts d'urgence et éviter les trajectoires dangereuses du robot.

La suite logique du projet consiste donc à connecter la perception Trace à la partie robotique. La position de balle estimée doit être convertie dans le repère du robot, filtrée, prédite, puis utilisée comme entrée de la politique de commande ou d'un contrôleur d'interception.

Bibliographie

- [1] Antonio Claudio De Sousa Dos Santos Filho, *Computer Vision, Reinforcement Learning and Skill Transfer in Robotic : Application to a Dynamic Task*, rapport de stage, document fourni.
- [2] CAP 20-25, *Le Centre International de Recherche “Innovative Transportation and Production Systems” - CIR ITPS*. <https://cap2025.fr/fr/challenge2>
- [3] CAP 20-25, *Useful information of CIR ITPS*. <https://cap2025.fr/.../useful-information-of-cir-ityps>
- [4] iniVation, *DVXplorer – Hardware documentation*, documentation officielle. <https://docs.inivation.com/hardware/current-products/dvxplorer.html>
- [5] iniVation, *Documentation officielle iniVation*. <https://docs.inivation.com/>
- [6] Open Robotics, *ROS 2 Documentation*, documentation officielle. <https://docs.ros.org/>
- [7] OpenCV, *Camera Calibration and 3D Reconstruction – calib3d module*, documentation officielle. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html
- [8] raylib, *raylib – A simple and easy-to-use library to enjoy videogames programming*, site officiel. <https://www.raylib.com/>
- [9] NVIDIA, *Isaac Sim – Robotics Simulation and Synthetic Data Generation*, site officiel. <https://developer.nvidia.com/isaac/sim>
- [10] NVIDIA, *Isaac Lab – Robot Learning Framework*, site officiel. <https://developer.nvidia.com/isaac/lab>
- [11] SensorsINI, *v2e : From video frames to DVS events*, dépôt GitHub. <https://github.com/SensorsINI/v2e>
- [12] Hu, Y., Liu, S.-C. et Delbruck, T., *v2e : From Video Frames to Realistic DVS Events*, CVPR Workshops, 2021. https://openaccess.thecvf.com/content/CVPR2021W/EventVision/html/Hu_v2e_From_Video_Frames_to_Realistic_DVS_Events_CVPRW_2021_paper.html
- [13] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. et Klimov, O., *Proximal Policy Optimization Algorithms*, arXiv, 2017. <https://arxiv.org/abs/1707.06347>